ATSA Presentation

Roll No D-003 & D013

Pradeep Talwelkar & Vishal Shah

Date:- 13th October 2024

Univariate Models

Micro Gas Turbine Electrical Energy Prediction Dataset from UC Irvine Machine Learning Repository

The dataset consists of three columns:

time: The time values.

input_voltage: A constant value (10) in this case.

el_power: The electric power measurements, which is the time series of interest for ARMA modeling.

https://archive.ics.uci.edu/dataset/994/micro+gas+turbine+electrical+en

1. MA Model

The code is designed to load a ex_1.csv dataset, visualize it, and fit a Moving Average (MA) model using the ARIMA framework from statsmodels. It then provides a forecast for a few steps ahead.

Importing necessary libraries:

pandas for handling the dataset. statsmodels.api for statistical modeling, especially for time series. matplotlib.pyplot for plotting and visualizing the data.

Dataset Loading:

The dataset ex_1.csv is in the current directory. Convert the 'time' column from seconds (assuming UNIX timestamp) to a datetime format and sets it as the index of the DataFrame.

Plotting:

Plots the column el_power (which represents electrical power) over time.

Fitting the MA model:

The ARIMA model with parameters (0, 0, q) represents an MA model because the ARIMA model (p, d, q) fits autoregressive (AR), integrated (I), and moving average (MA) components. q=3 is specified, which

means a moving average model with 3 lagged terms.

Summary:

Outputs the summary of the model fit, which provides statistical details.

Forecasting:

Uses the forecast method to predict the next 10 time steps.

In [1]:
```
# Required libraries
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv('ex_1.csv')

# Setting the time column as index
data['time'] = pd.to_datetime(data['time'], unit='s')  # Assuming the 'time' column is in sec
data.set_index('time', inplace=True)

# Visualize the data
plt.figure(figsize=(10, 5))
plt.plot(data['el_power'])
plt.title('Electrical Power Over Time')
plt.show()

# Fitting an MA model
# Note: q is the window size for the moving average
q = 3  # You can adjust the number of lags
model = sm.tsa.ARIMA(data['el_power'], order=(0, 0, q))  # (p=0, d=0, q=MA order)
results = model.fit()

# Summary of the model
print(results.summary())

# Forecasting the next few steps
forecast_steps = 10
forecast = results.forecast(steps=forecast_steps)

# Since the forecast is likely a NumPy array, we can print it directly
print(f"Forecasted values: {forecast}")


# Comparison Parameters
from sklearn.metrics import mean_squared_error
import numpy as np

# AIC and BIC from the model results
aic = results.aic
bic = results.bic

# Calculate Root Mean Squared Error (RMSE)
# Getting the model residuals (errors)
residuals = results.resid

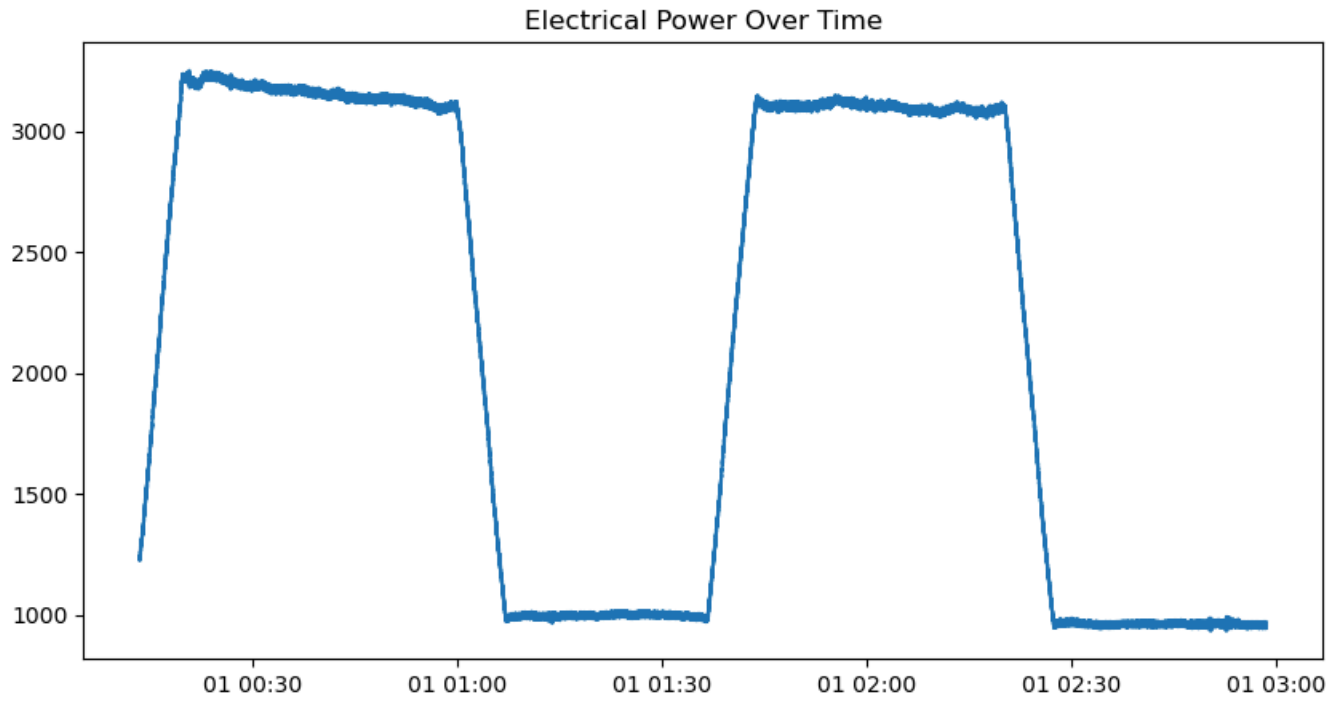
# Compute RMSE from residuals
rmse = np.sqrt(mean_squared_error(data['el_power'], results.fittedvalues))

# Printing the comparison metrics
print(f"AIC: {aic}")
print(f"BIC: {bic}")
print(f"RMSE: {rmse}")

# Mean Absolute Error (MAE)
mae = np.mean(np.abs(residuals))
```

```
print(f"MAE: {mae}")

# Mean Absolute Percentage Error (MAPE)
mape = np.mean(np.abs(residuals / data['el_power'])) * 100
print(f"MAPE: {mape}%")
```




```
C:\Users\Lenovo\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:471: ValueWarni
ng: A date index has been provided, but it has no associated frequency information and so wil
l be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
C:\Users\Lenovo\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:471: ValueWarni
ng: A date index has been provided, but it has no associated frequency information and so wil
l be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
C:\Users\Lenovo\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:471: ValueWarni
ng: A date index has been provided, but it has no associated frequency information and so wil
l be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                el_power   No. Observations:                 9920
Model:                 ARIMA(0, 0, 3)   Log Likelihood              -71145.955
Date:                Sat, 05 Oct 2024   AIC                         142301.911
Time:                        15:25:48   BIC                         142337.923
Sample:                             0   HQIC                        142314.106
                                 - 9920
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const       2162.4179     19.001    113.804      0.000    2125.176    2199.660
ma.L1          1.9541      0.010    191.345      0.000       1.934       1.974
ma.L2          1.9532      0.028     70.636      0.000       1.899       2.007
ma.L3          0.9396      0.018     52.582      0.000       0.905       0.975
sigma2      9.849e+04      0.039   2.49e+06      0.000    9.85e+04    9.85e+04
===================================================================================
Ljung-Box (L1) (Q):                  60.63   Jarque-Bera (JB):               320.02
Prob(Q):                              0.00   Prob(JB):                         0.00
Heteroskedasticity (H):               4.78   Skew:                            -0.44
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.11
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 3.74e+21. Standard
errors may be unstable.
Forecasted values: 9920     623.015624
9921     390.526938
9922    1362.916210
9923    2162.417880
9924    2162.417880
9925    2162.417880
9926    2162.417880
9927    2162.417880
9928    2162.417880
9929    2162.417880
Name: predicted_mean, dtype: float64
AIC: 142301.91097456167
BIC: 142337.92251556306
RMSE: 315.06586975351
MAE: 251.079159122662
MAPE: 16.7205431178981%
C:\Users\Lenovo\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:834: ValueWarni
ng: No supported index is available. Prediction results will be given with an integer index b
eginning at `start`.
  return get_prediction_index(
```

For MA Model

AIC: 142301.91097456167

BIC: 142337.92251556306

RMSE: 315.06586975351

MAE: 251.079159122662

MAPE: 16.7205431178981 %

2. AR Model

This code performs a time series analysis and forecasting using the ARIMA (Autoregressive Integrated Moving Average) (1, 0, 0) model or AR(1) Model

AR (Autoregressive) part:

The "1" in ARIMA(1, 0, 0) indicates that the model uses one lag of the dependent variable (the time series) to predict its current value. This means that the current value of the series is regressed on its immediately preceding value.

I (Integrated) part:

The "0" indicates that no differencing of the time series is applied. Differencing is typically used to make a non-stationary series stationary.

MA (Moving Average) part:

The "0" means that no lagged forecast errors are included in the model. So, it only relies on the past values of the series itself for predictions.

Characteristics of the AR Model The AR model captures the relationship between an observation and a number of lagged observations (previous values).

For an AR(1) model, the equation can be represented as:

$Xt = c + \phi 1 Xt-1 + \epsilon t$

where:

Xt is the current value of the series,

c is a constant,

$\phi 1$ is the coefficient for the lagged term,

$Xt-1$ is the previous value of the series,

ϵt is the white noise error term.

1. Importing Libraries

The necessary libraries for data manipulation (pandas, numpy), visualization (matplotlib), and time series analysis (statsmodels) are imported.

2. Loading the Dataset

The dataset is loaded from a CSV file named ex_1.csv. You should ensure this file is in the same directory or provide the correct path.

3. Plotting the Raw Data

The first visualization shows the electrical power consumption over time, helping you to identify any trends or seasonal patterns.

4. ADF Test for Stationarity

The Augmented Dickey-Fuller (ADF) test checks whether the time series is stationary. The ADF statistic, p-value, and critical values are printed. A low p-value (typically less than 0.05) indicates that the time series is stationary.

5. ACF and PACF Plots

The Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots help determine the appropriate values for the AR (p) and MA (q) components of the ARIMA model. ACF shows the correlation of the series with its own lagged values. PACF shows the correlation of the series with its own lagged values after removing the effects of shorter lags.

6. Fitting the AR(1) Model

An ARIMA(1, 0, 0) model (which is essentially an AR(1) model) is fitted to the series. The summary of the model provides insights into the coefficients and their significance.

7. Residual Analysis The residuals of the fitted model are plotted to check for randomness (ideally, they should resemble white noise). ACF and PACF plots of the residuals are also generated to check for any remaining autocorrelation.

8. Ljung-Box Test

The Ljung-Box test checks if there are significant autocorrelations in the residuals. If the p-value is high, it suggests that there is no autocorrelation in the residuals, supporting the adequacy of the model.

9. Forecasting The next 10 points of the time series are forecasted using the fitted AR(1) model. The forecast and its confidence intervals are plotted against the observed data.

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_pacf, plot_acf
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
import warnings
from sklearn.metrics import mean_squared_error, mean_absolute_error

warnings.filterwarnings('ignore')

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Extract the relevant time series (el_power column)
el_power_series = data['el_power']

# Step 1: Plot the raw data to visualize the time series
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Electrical Power')
plt.title('Electrical Power over Time')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.grid(True)
plt.legend()
plt.show()

# Step 2: Perform the ADF test to check stationarity
adf_test = adfuller(el_power_series)

print(f"ADF Statistic: {adf_test[0]}")
print(f"p-value: {adf_test[1]}")
print("Critical Values:")
for key, value in adf_test[4].items():
    print(f"   {key}: {value}")

# Step 3: Plot ACF and PACF to identify AR and MA terms
plt.figure(figsize=(10, 5))
```

```
plot_pacf(el_power_series, lags=40)
plt.title('Partial Autocorrelation Function (PACF)')
plt.show()

plt.figure(figsize=(10, 5))
plot_acf(el_power_series, lags=40)
plt.title('Autocorrelation Function (ACF)')
plt.show()

# Step 4: ARIMA model for AR(1)
# ARIMA(1, 0, 0) represents AR(1) with no differencing (d=0) and no moving average terms (q=0

p, d, q = 1, 0, 0

# Fit AR(1) model
model = ARIMA(el_power_series, order=(p, d, q))
model_fit = model.fit()

# Step 5: Summary of the ARIMA model
print(model_fit.summary())

# AIC and BIC
aic = model_fit.aic
bic = model_fit.bic

print(f"AIC: {aic}")
print(f"BIC: {bic}")

# Step 6: Calculate RMSE, MAE, MAPE for model
# Get predicted values for in-sample (fitted) data
fitted_values = model_fit.fittedvalues

# RMSE
rmse = np.sqrt(mean_squared_error(el_power_series, fitted_values))
print(f"RMSE: {rmse}")

# MAE
mae = mean_absolute_error(el_power_series, fitted_values)
print(f"MAE: {mae}")

# MAPE (Mean Absolute Percentage Error)
mape = np.mean(np.abs((el_power_series - fitted_values) / el_power_series)) * 100
print(f"MAPE: {mape}")

# Step 7: Plot the residuals to check for randomness
plt.figure(figsize=(10, 5))
plt.plot(model_fit.resid)
plt.title('Residuals of the AR(1) Model')
plt.grid(True)
plt.show()

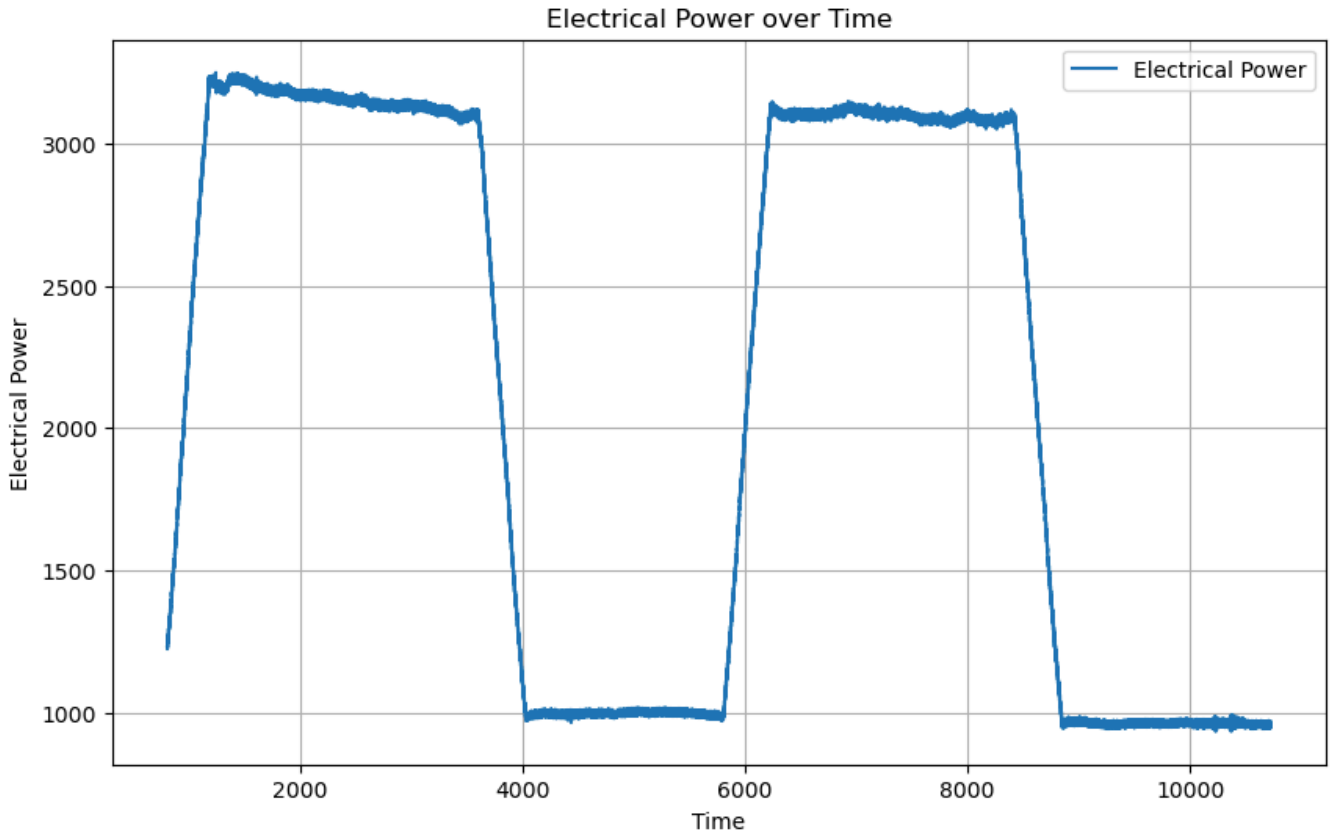
# Plot ACF and PACF of residuals to check for autocorrelation
plot_acf(model_fit.resid, lags=40)
plt.title('ACF of Residuals')
plt.show()

plot_pacf(model_fit.resid, lags=40)
plt.title('PACF of Residuals')
plt.show()

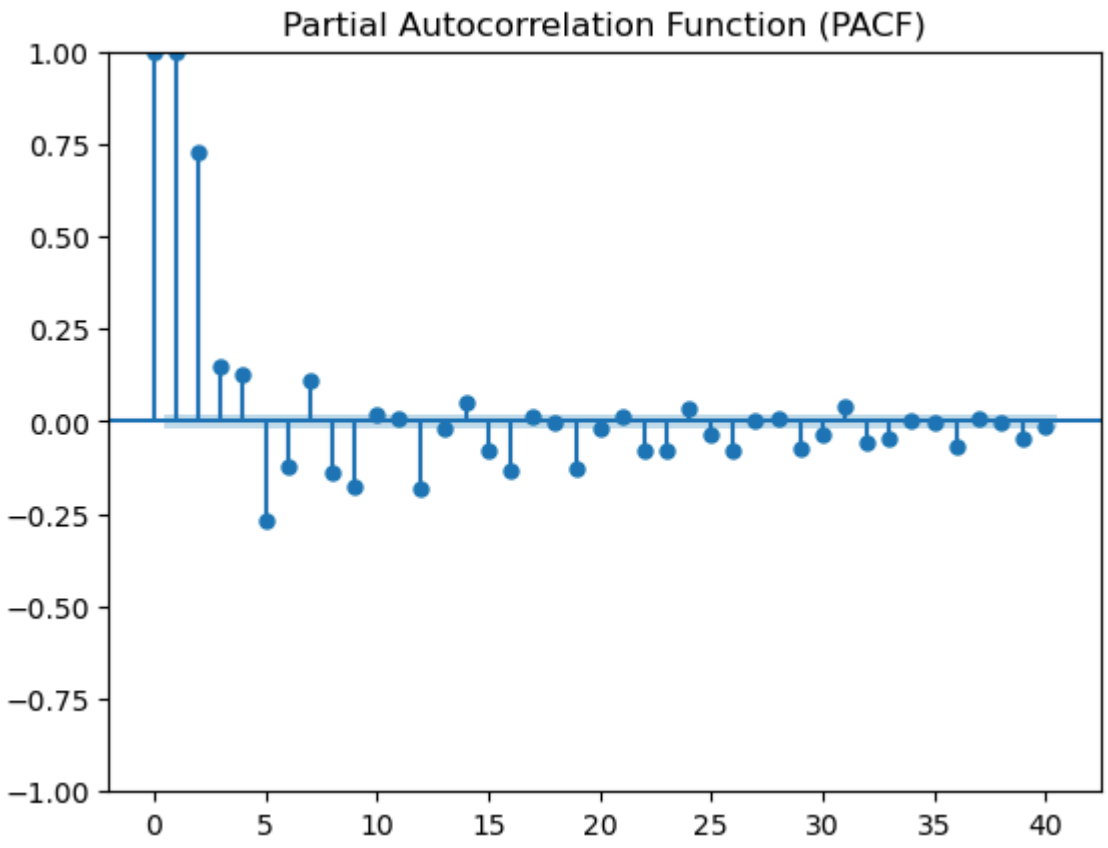
# Step 8: Ljung-Box test for residual autocorrelation
from statsmodels.stats.diagnostic import acorr_ljungbox
lb_test = acorr_ljungbox(model_fit.resid, lags=[10], return_df=True)
print("Ljung-Box Test Results:")
print(lb_test)

# Step 9: Forecasting the next 10 points using the fitted AR(1) model
forecast = model_fit.get_forecast(steps=10)
forecast_ci = forecast.conf_int()
```

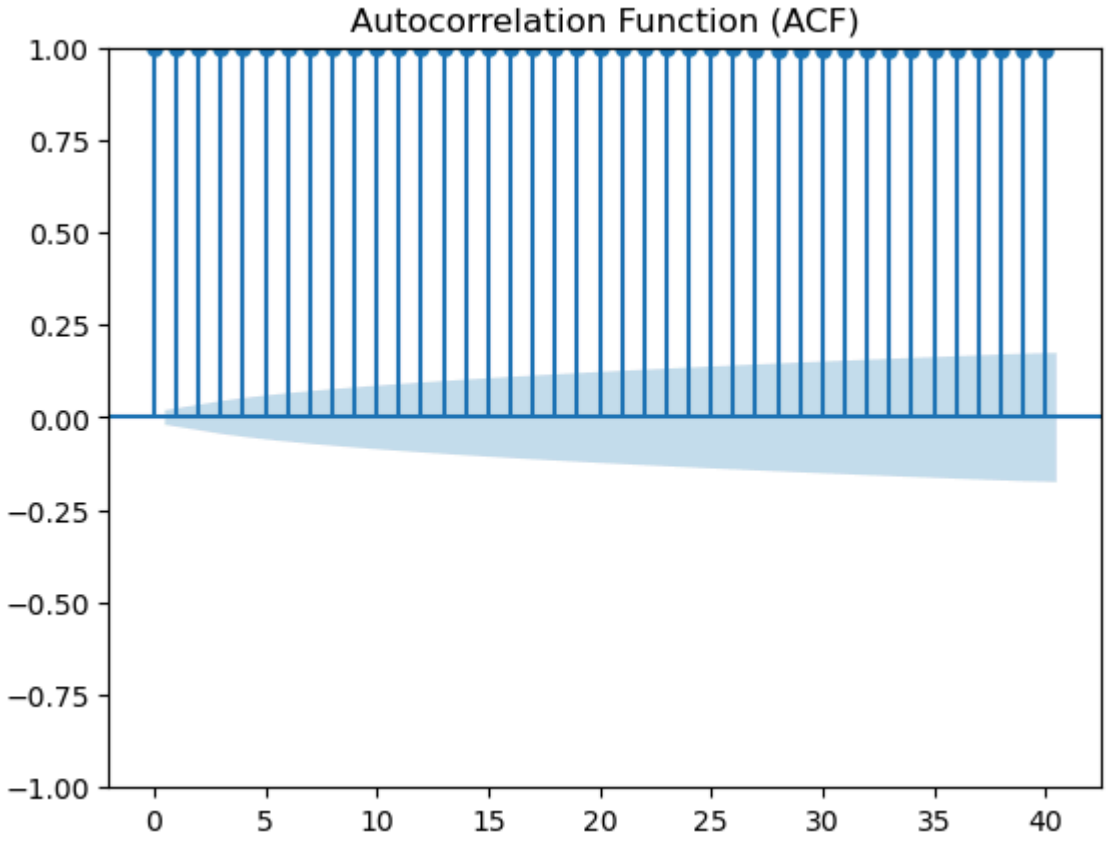
```
# Plot the forecasted values
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Observed Data')
plt.plot(np.arange(len(data), len(data) + 10), forecast.predicted_mean, label='Forecast')
plt.fill_between(np.arange(len(data), len(data) + 10),
                 forecast_ci.iloc[:, 0], forecast_ci.iloc[:, 1], color='gray', alpha=0.3)
plt.title('Forecast of Electrical Power (AR(1) Model)')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.legend()
plt.grid(True)
plt.show()
```




```
ADF Statistic: -2.88003477482922
p-value: 0.04771942827842128
Critical Values:
   1%: -3.4310119774951406
   5%: -2.861832554286939
   10%: -2.566925721514317
<Figure size 1000x500 with 0 Axes>
```



<Figure size 1000x500 with 0 Axes>



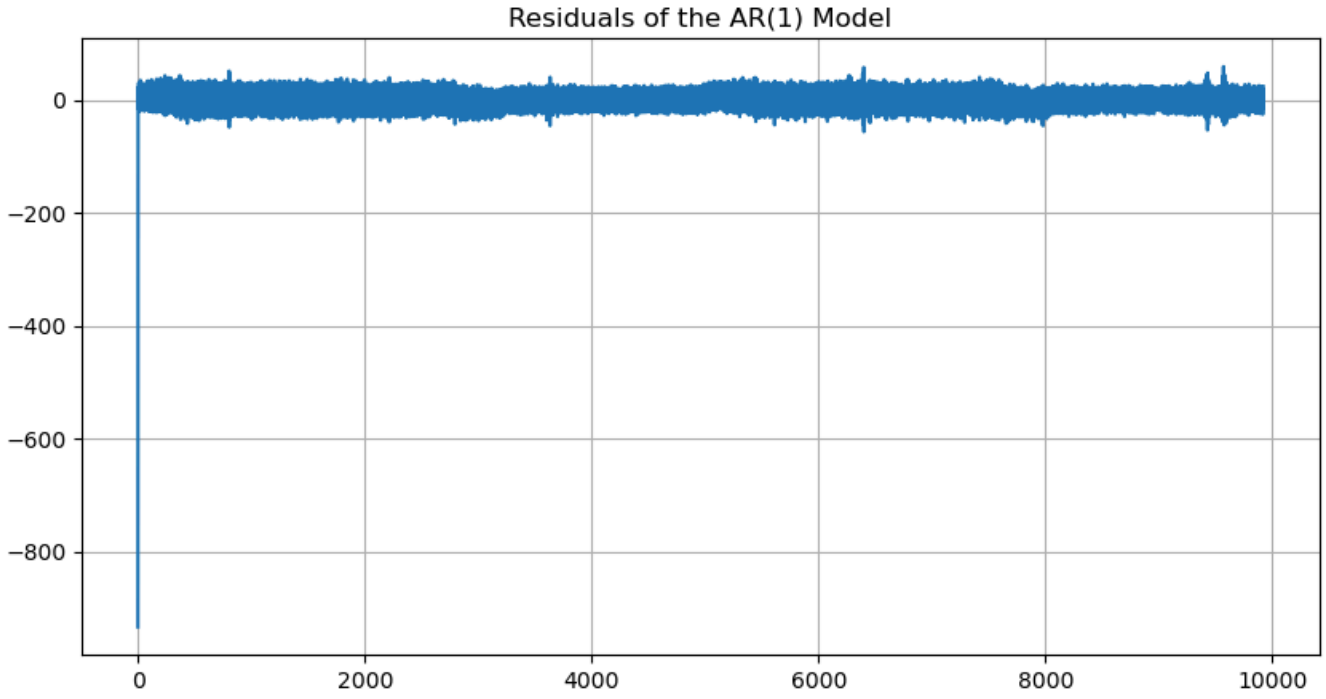
SARIMAX Results

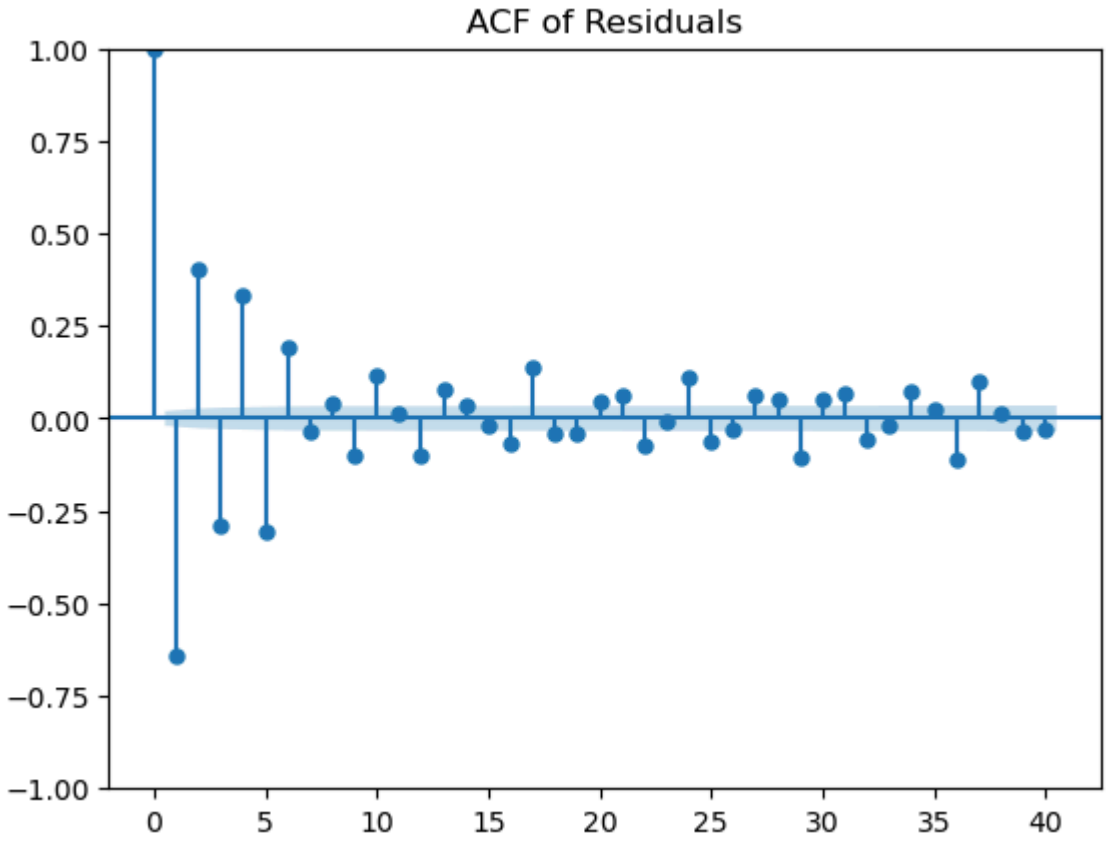
Dep. Variable:	el_power	No. Observations:	9920
Model:	ARIMA(1, 0, 0)	Log Likelihood	-43787.749
Date:	Sat, 05 Oct 2024	AIC	87581.499
Time:	15:25:50	BIC	87603.106
Sample:	0	HQIC	87588.816
	- 9920		
Covariance Type:	opg		

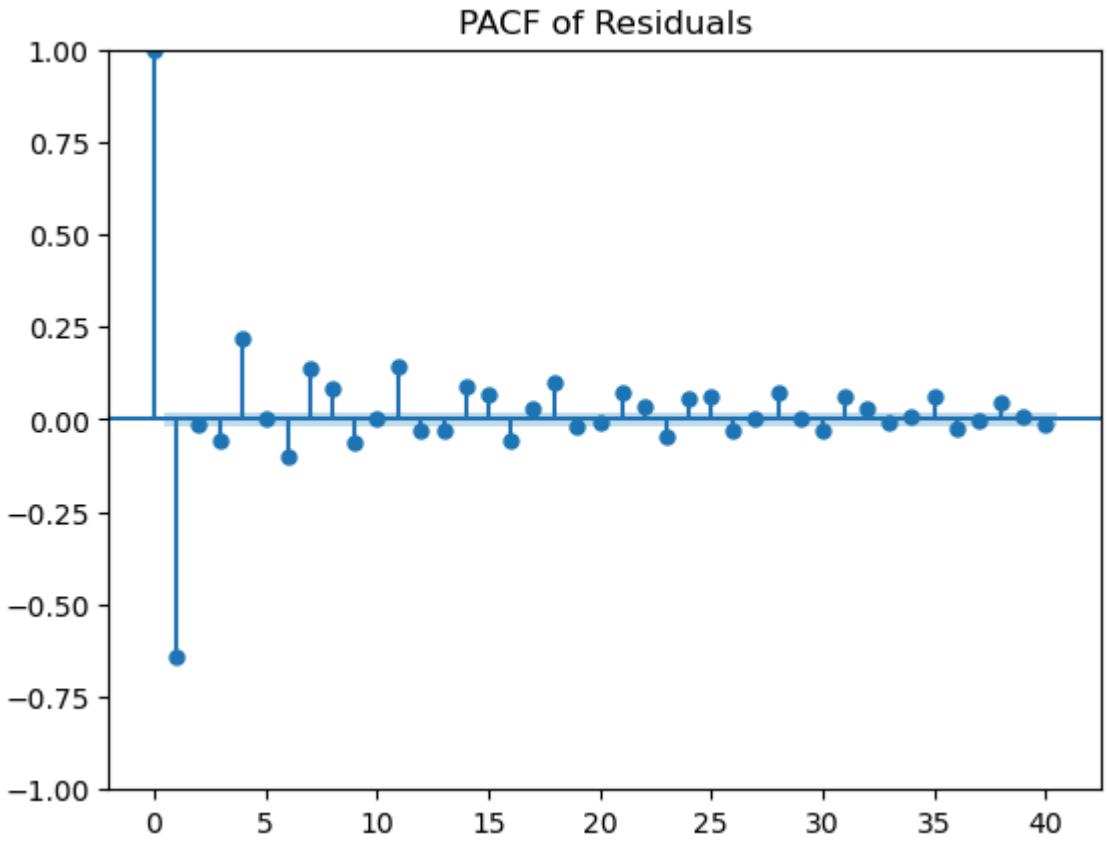
	coef	std err	z	P>|z|	[0.025	0.975]
const	2162.4279	906.568	2.385	0.017	385.587	3939.268
ar.L1	0.9998	0.000	4758.680	0.000	0.999	1.000
sigma2	399.2225	11.180	35.709	0.000	377.310	421.135

Ljung-Box (L1) (Q):	6138.06	Jarque-Bera (JB):	912.13
Prob(Q):	0.00	Prob(JB):	0.00
Heteroskedasticity (H):	0.78	Skew:	0.01
Prob(H) (two-sided):	0.00	Kurtosis:	1.51

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
AIC: 87581.4989274149
BIC: 87603.10585201574
RMSE: 22.069273924782024
MAE: 18.65331198633499
MAPE: 1.0641399794449966

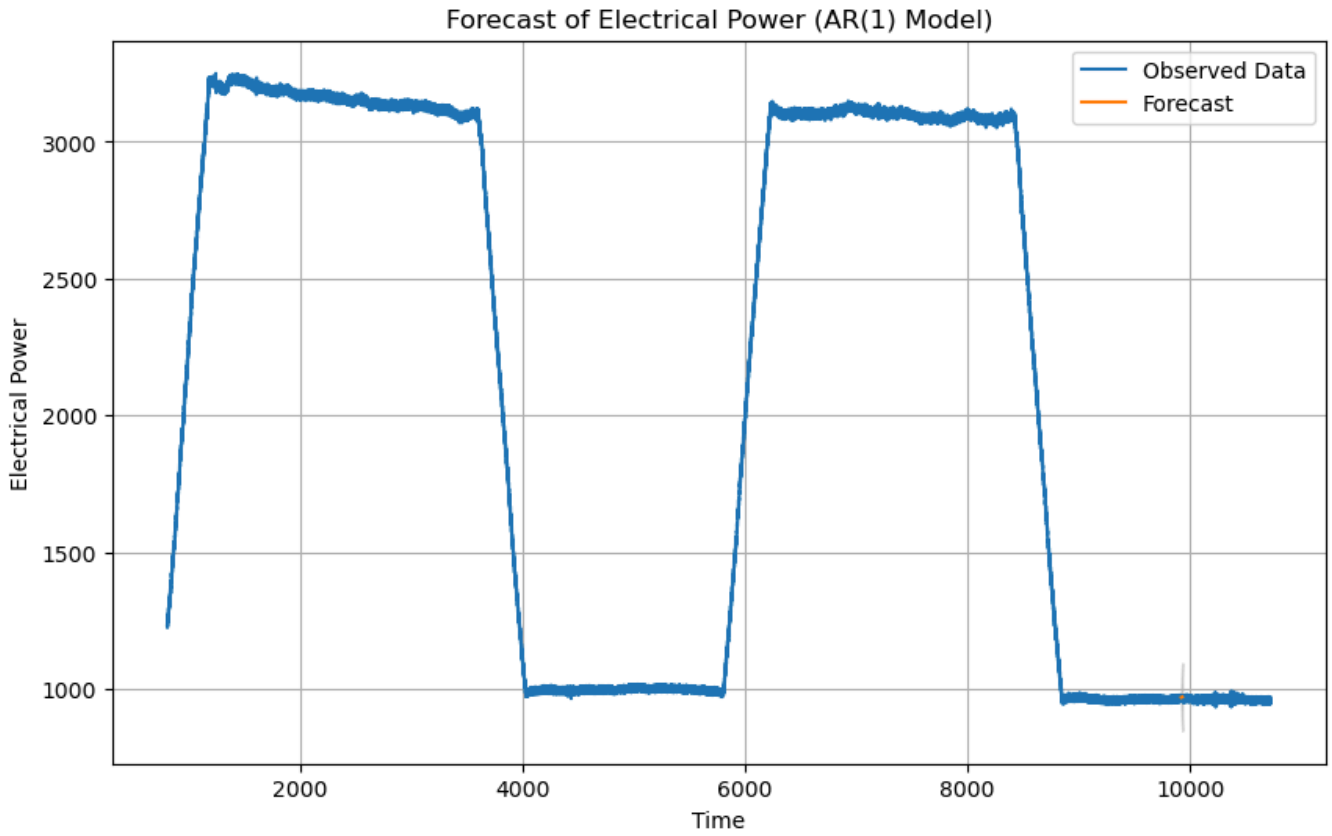






Ljung-Box Test Results:

	lb_stat	lb_pvalue
10	9216.223299	0.0



For AR Model

AIC: 87581.4989274149

BIC: 87603.10585201574

RMSE: 22.069273924782024

MAE: 18.65331198633499

MAPE: 1.0641399794449966 %

3. AUTOREGRESSIVE MOVING AVERAGE (ARMA) MODEL

This code performs a time series analysis and forecasting using the ARIMA model, specifically an ARIMA(2, 0, 2) model.

AR: Autoregressive part, represented by the parameter p.

p=2 means that the model uses 2 lagged values of the time series.

I: Integrated part, represented by the parameter d.

d=0 indicates that no differencing is applied, which means the series is assumed to be stationary._

This effectively makes the model an ARMA model because there is no differencing involved.

MA: Moving Average part, represented by the parameter q.

q=2 means the model uses 2 lagged forecast errors.

Since the model is defined with d=0, it is an ARMA(2, 2) model, which is a special case of the ARIMA model.

ARIMA can be thought of as a generalization of ARMA, allowing for non-stationary time series through differencing when d>0.

1. Importing Libraries

We import necessary libraries for data manipulation (pandas, numpy), plotting (matplotlib.pyplot), and statistical modeling (statsmodels). The warning filter is applied to suppress warnings that may arise during the execution of your code.

2. Data Loading

We load the dataset and extract the el_power column for analysis.

3. Visualization

We visualize the time series data and generate the Partial Autocorrelation Function (PACF) and Autocorrelation Function (ACF) plots. This helps in understanding the temporal structure of the data.

4. ARIMA Model Fitting

We define the ARIMA model with parameters p=2, d=0, and q=2, and fit the model to the el_power data. We print the summary of the fitted model, which provides useful diagnostics.

5. Residual Analysis

We plot the residuals to check if they behave like white noise. Then we plot the ACF and PACF of the residuals to ensure no significant autocorrelation remains. This is crucial for validating the model.

6. Ljung-Box Test

We apply the Ljung-Box test to statistically confirm the absence of autocorrelation in the residuals, which is important for the validity of our ARIMA model.

7. Forecasting

We forecast the next 10 time points and visualize both the observed and forecasted data. The confidence intervals are also plotted to provide insight into the uncertainty of the forecasts.

In [3]:
```
# Importing Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_pacf, plot_acf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
import warnings

warnings.filterwarnings('ignore')

# Data Loading:

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Visualization:
```

```
# Plot the raw data
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Electrical Power')
plt.title('Electrical Power over Time')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.grid(True)
plt.legend()
plt.show()

# Simulating PACF and ACF plots
plt.figure(figsize=(10, 5))
plot_pacf(data['el_power'], lags=40, method='ywm')
plt.title('Partial Autocorrelation Function (PACF)')
plt.show()

plt.figure(figsize=(10, 5))
plot_acf(data['el_power'], lags=40)
plt.title('Autocorrelation Function (ACF)')
plt.show()

# ARIMA Model:

# Fit ARMA/ARIMA model to the el_power time series data
p, d, q = 2, 0, 2  # ARIMA(2, 0, 2) model

model = ARIMA(data['el_power'], order=(p, d, q))
model_fit = model.fit()

# Summary of the model
print(model_fit.summary())

# Residual Analysis:

# Plot the residuals
plt.figure(figsize=(10, 5))
plt.plot(model_fit.resid)
plt.title('Residuals of the ARIMA Model')
plt.grid(True)
plt.show()

# Plot ACF and PACF of residuals
plot_acf(model_fit.resid, lags=40)
plt.title('ACF of Residuals')
plt.show()

plot_pacf(model_fit.resid, lags=40, method='ywm')
plt.title('PACF of Residuals')
plt.show()

# Ljung-Box Test:

lb_test = acorr_ljungbox(model_fit.resid, lags=[10], return_df=True)
print("Ljung-Box Test Results:")
print(lb_test)

# Forecasting:

forecast = model_fit.get_forecast(steps=10)
forecast_ci = forecast.conf_int()

# Plot the forecasted values
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Observed Data')
plt.plot(np.arange(data['time'].iloc[-1], data['time'].iloc[-1] + 10), forecast.predicted_mea
plt.fill_between(np.arange(data['time'].iloc[-1], data['time'].iloc[-1] + 10),
                 forecast_ci.iloc[:, 0], forecast_ci.iloc[:, 1], color='gray', alpha=0.3)
plt.title('Forecast of Electrical Power')
plt.xlabel('Time')
```

```
plt.ylabel('Electrical Power')
plt.legend()
plt.grid(True)
plt.show()

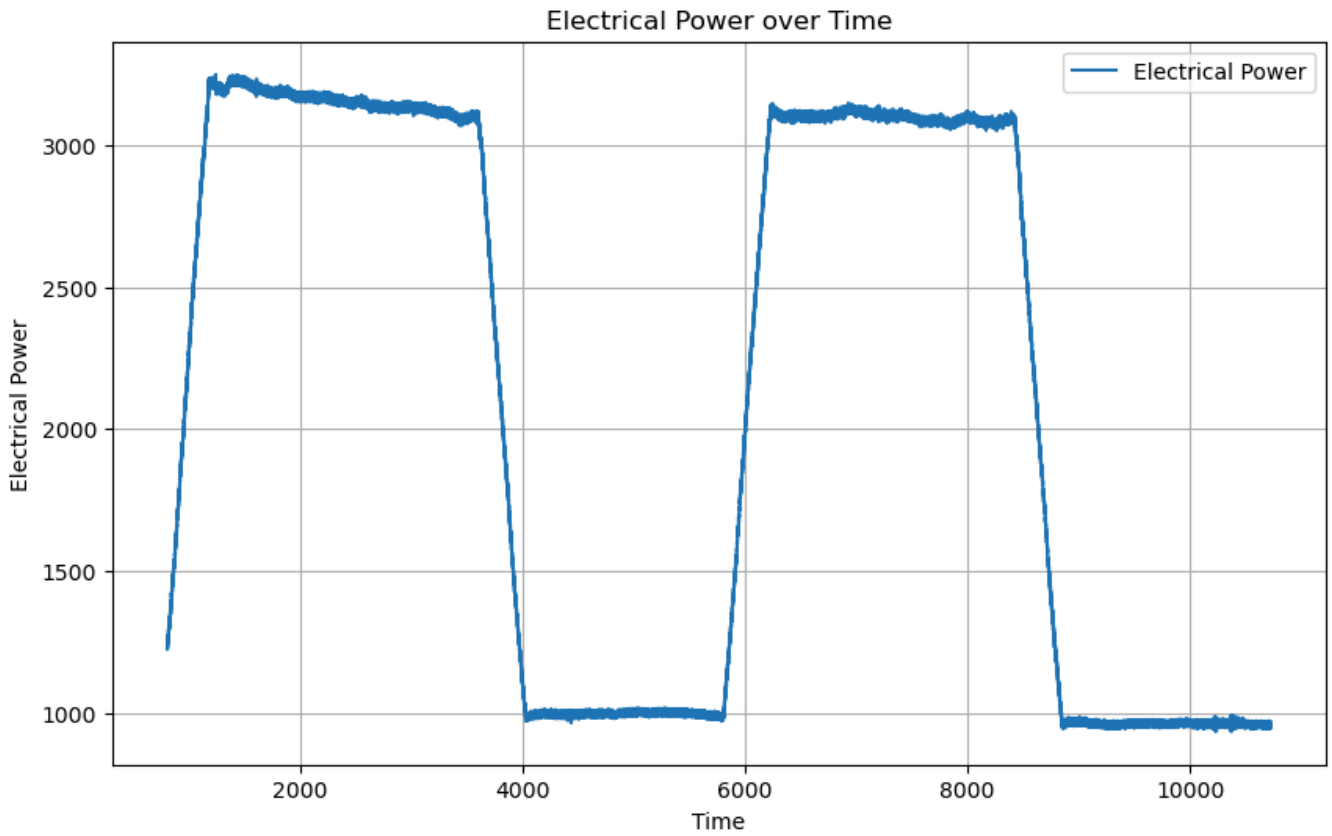
# Calculating AIC and BIC
aic = model_fit.aic
bic = model_fit.bic
print(f"AIC: {aic}")
print(f"BIC: {bic}")

# Calculating RMSE, MAE, and MAPE
# Forecasting using the last known values for comparison
actual = data['el_power'].values[-10:]  # last 10 observed values
predicted = forecast.predicted_mean.values  # forecasted values

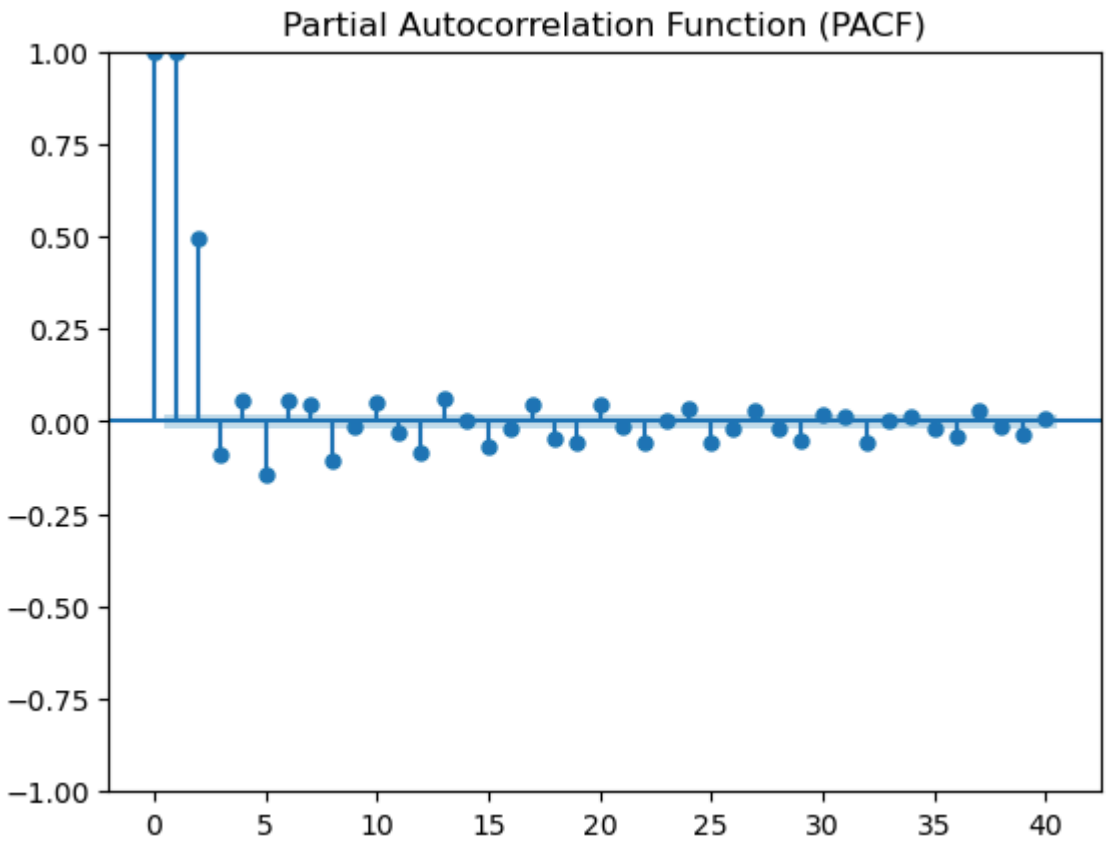
# Calculate RMSE
rmse = np.sqrt(np.mean((actual - predicted) ** 2))
print(f"RMSE: {rmse}")

# Calculate MAE
mae = np.mean(np.abs(actual - predicted))
print(f"MAE: {mae}")

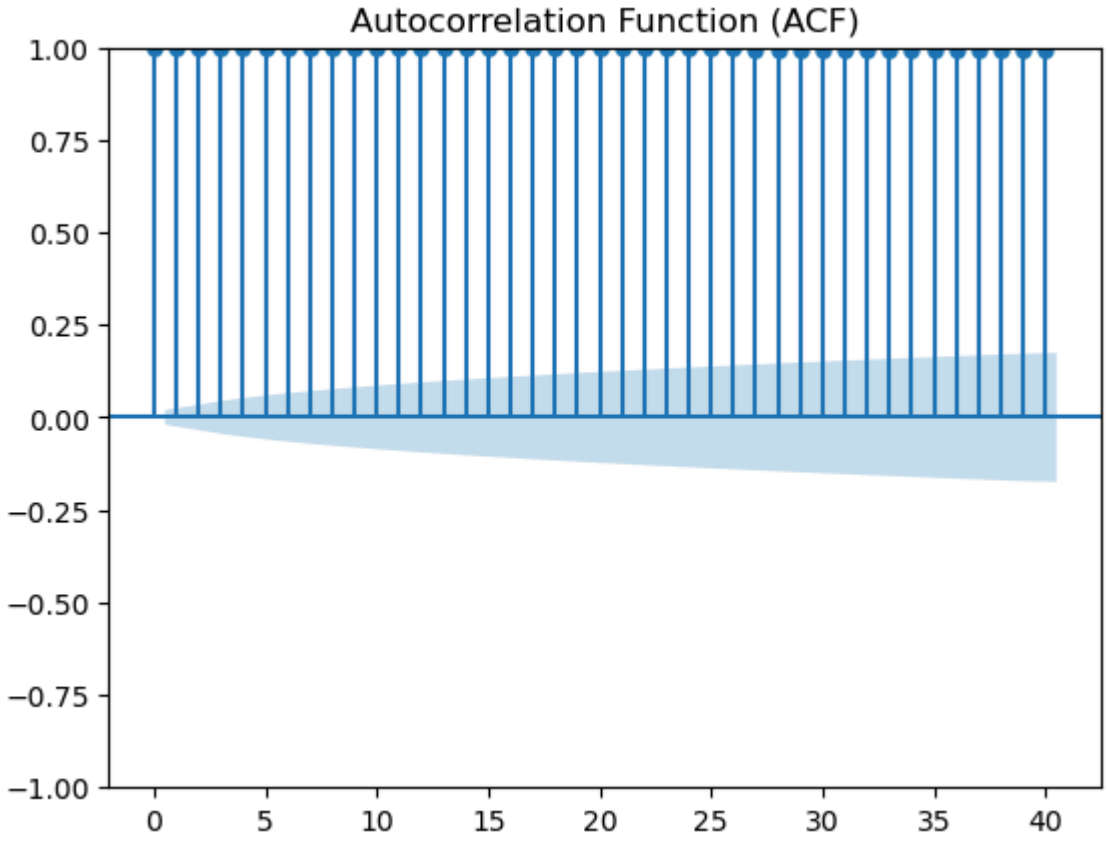
# Calculate MAPE
mape = np.mean(np.abs((actual - predicted) / actual)) * 100
print(f"MAPE: {mape}%")
```




```
<Figure size 1000x500 with 0 Axes>
```



<Figure size 1000x500 with 0 Axes>



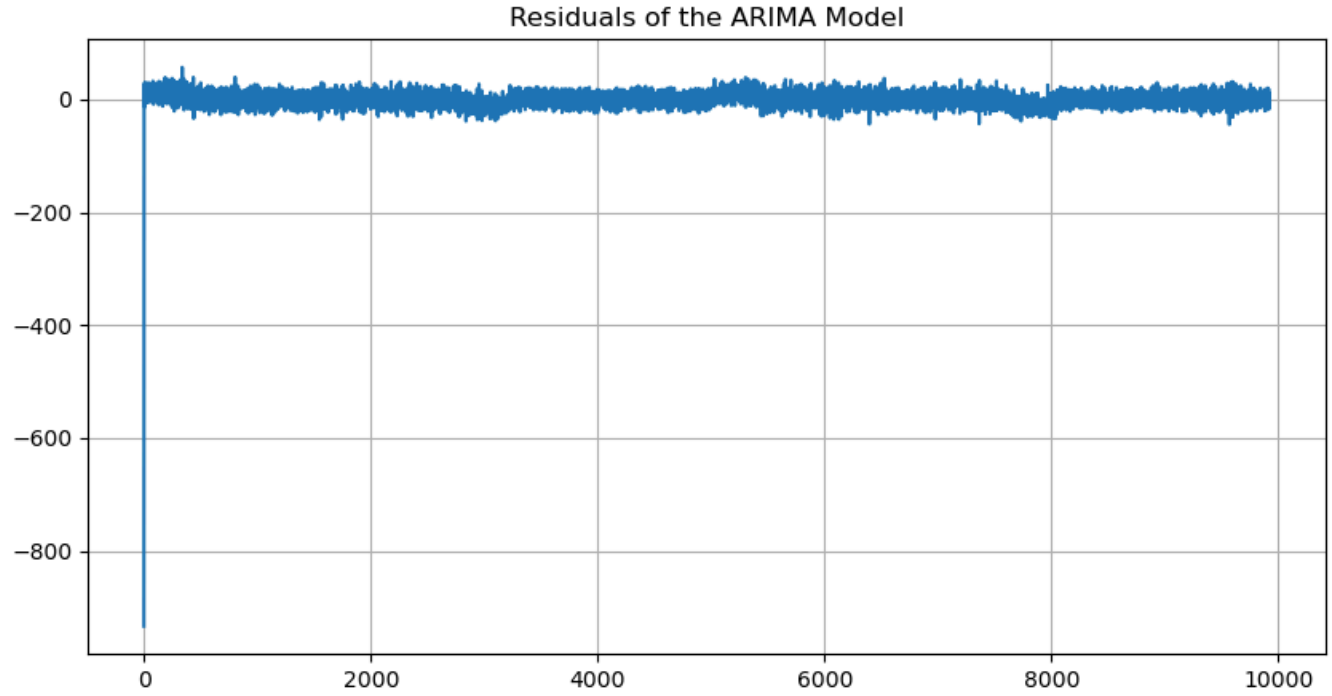
SARIMAX Results

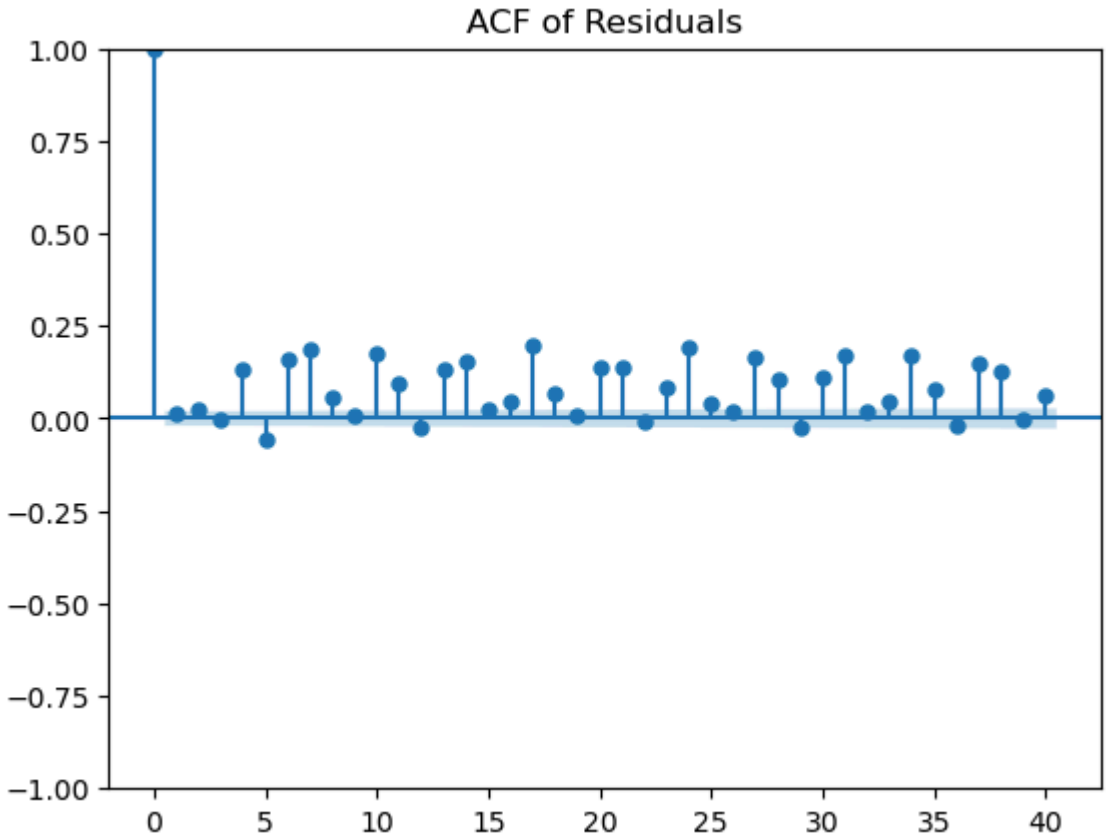
Dep. Variable:	el_power	No. Observations:	9920
Model:	ARIMA(2, 0, 2)	Log Likelihood	-37774.344
Date:	Sat, 05 Oct 2024	AIC	75560.689
Time:	15:26:09	BIC	75603.903
Sample:	0 - 9920	HQIC	75575.322
Covariance Type:	opg		

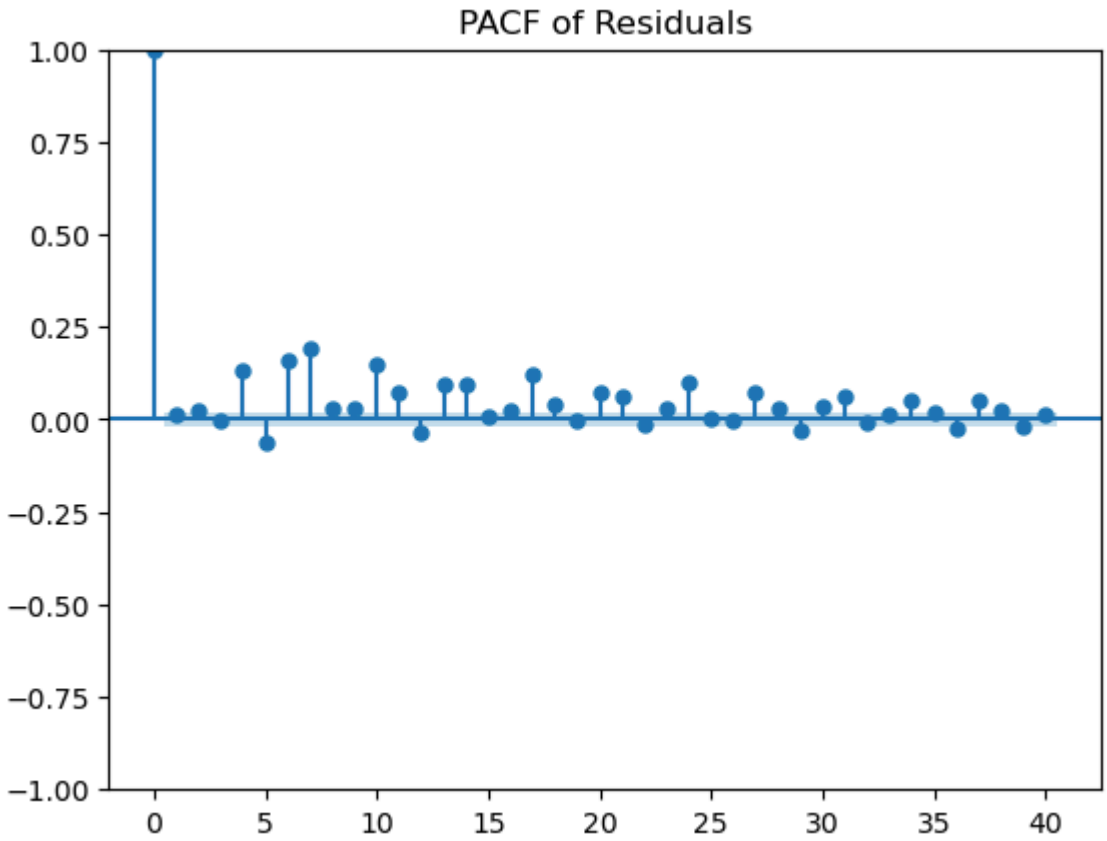
	coef	std err	z	P>|z|	[0.025	0.975]
const	2162.3684	474.199	4.560	0.000	1232.956	3091.781
ar.L1	0.7264	0.016	45.961	0.000	0.695	0.757
ar.L2	0.2735	0.016	17.306	0.000	0.243	0.304
ma.L1	-0.9386	0.012	-79.238	0.000	-0.962	-0.915
ma.L2	0.5381	0.010	52.609	0.000	0.518	0.558
sigma2	118.1314	1.791	65.970	0.000	114.622	121.641

Ljung-Box (L1) (Q):	4.53	Jarque-Bera (JB):	43.22
Prob(Q):	0.03	Prob(JB):	0.00
Heteroskedasticity (H):	0.80	Skew:	0.04
Prob(H) (two-sided):	0.00	Kurtosis:	3.31

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

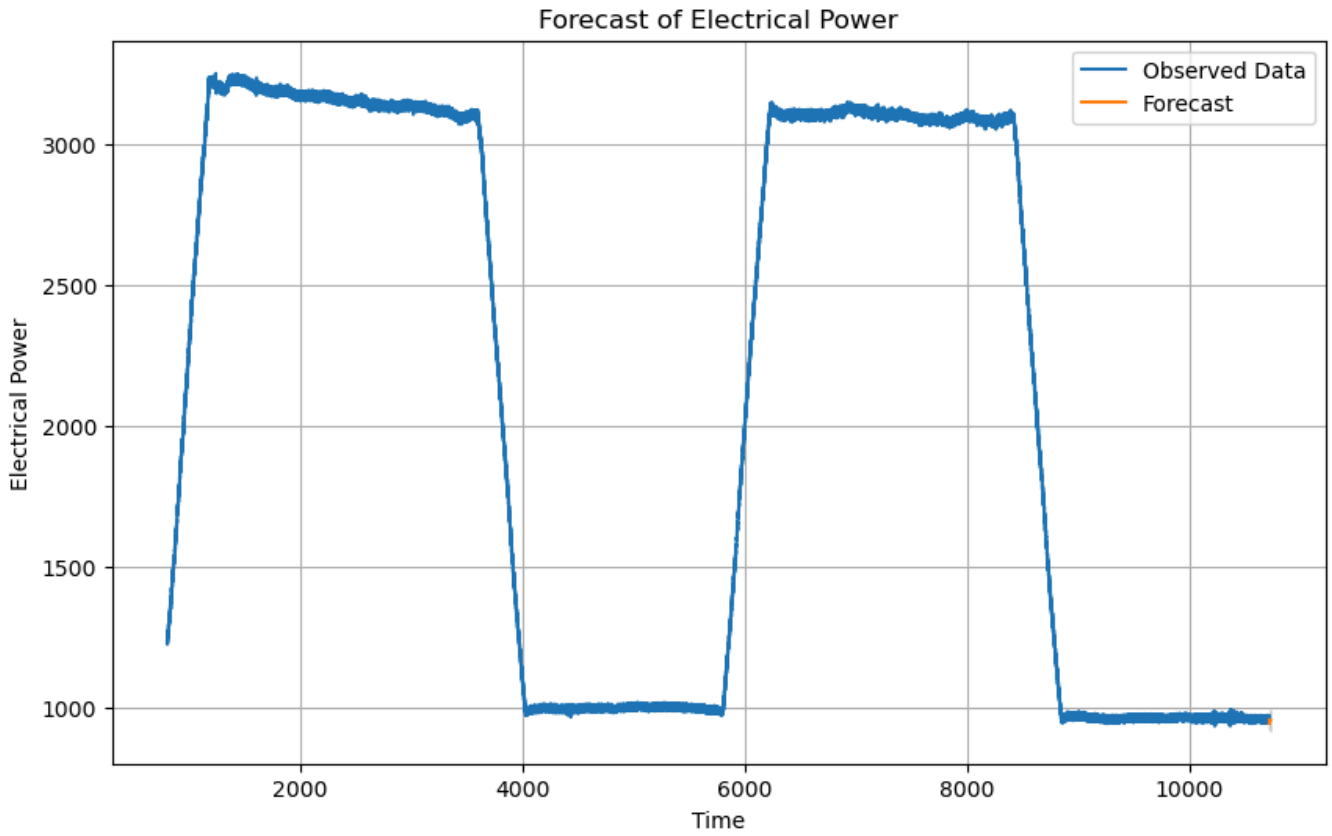






Ljung-Box Test Results:

	lb_stat	lb_pvalue
10	1144.858846	1.124027e-239




```
AIC: 75560.6889696246
BIC: 75603.90281882627
RMSE: 9.533227610716276
MAE: 8.435868927015713
MAPE: 0.8798867618262285%
```

For ARMA Model

AIC: 75560.6889696246

BIC: 75603.90281882627

RMSE: 9.533227610716276

MAE: 8.435868927015713

MAPE: 0.8798867618262285 %

4. AUTOREGRESSIVE INTEGRATED MOVING AVERAGE (ARIMA) MODEL

To modify the code for ARIMA (Autoregressive Integrated Moving Average) instead of ARMA, we need to introduce differencing, which is the "I" component in ARIMA.

The order of differencing is controlled by the d parameter in ARIMA(p, d, q), where:

p: Number of autoregressive terms.

d: Number of differences needed to make the series stationary.

q: Number of moving average terms.

Choosing the right parameters (p,d,q) for an ARIMA model is crucial for obtaining a good fit and accurate forecasts.

1. Determine d (Differencing Order):

Stationarity Check: The first step is to determine if the time series is stationary. We can perform the Augmented Dickey-Fuller (ADF) test in the code. If the p-value is greater than 0.05, the series is non-stationary, and we should use differencing: d=1 if the series requires first differencing. You can also apply seasonal differencing if you observe seasonal patterns. If the series is stationary (p-value ≤ 0.05), then d=0.

2. Determine p (Autoregressive Order):

PACF Plot: Use the Partial Autocorrelation Function (PACF) plot to identify the order of the autoregressive (AR) component. Look for the lag at which the PACF plot cuts off. The point at which the PACF drops to zero (or becomes negligible) indicates the potential value for p.

3. Determine q (Moving Average Order):

ACF Plot: Use the Autocorrelation Function (ACF) plot to identify the order of the moving average (MA) component. Similar to PACF, look for the lag where the ACF cuts off. The lag at which the ACF drops to zero indicates the potential value for q.

4. Using Information Criteria:

Consider using AIC (Akaike Information Criterion) or BIC (Bayesian Information Criterion) for model selection. Lower values of AIC/BIC indicate a better model fit.

Importing Libraries:

Essential libraries such as pandas, numpy, matplotlib, and statsmodels are imported for data manipulation, visualization, and time series analysis.

Loading the Dataset:

The dataset is loaded from a CSV file into a Pandas DataFrame. You extract the el_power column as the time series to be analyzed.

Visualizing the Time Series:

You plot the raw electrical power data over time to visually inspect trends and patterns.

Stationarity Check with ADF Test:

The Augmented Dickey-Fuller (ADF) test is performed to check the stationarity of the time series. If the p-value is greater than 0.05, the series is deemed non-stationary.

ACF and PACF Plots:

You plot the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) to identify potential values for the autoregressive (AR) and moving average (MA) components of the ARIMA model.

Differencing:

If the ADF test indicates non-stationarity, you apply differencing (set d=1d=1) to the time series.

Fitting the ARIMA Model:

We fit the ARIMA model with specified p and q values (2 in this case) and the determined d.__

Model Summary:

The summary of the fitted ARIMA model is printed, providing details about the estimated parameters and diagnostics.

Residual Analysis:

We plot the residuals to check for patterns. Ideally, residuals should resemble white noise.

ACF and PACF of Residuals:

ACF and PACF plots of residuals are generated to confirm that no autocorrelation remains in the residuals.

Forecasting Future Values:

Finally, we forecast the next 10 time steps and plot the historical data alongside the forecasted values.

In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_pacf, plot_acf
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
import warnings

warnings.filterwarnings('ignore')

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Extract the relevant time series (el_power column)
el_power_series = data['el_power']

# Step 1: Plot the raw data to visualize the time series
plt.figure(figsize=(10, 6))
plt.plot(data['time'], el_power_series, label='Electrical Power')
plt.title('Electrical Power over Time')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.grid(True)
plt.legend()
plt.show()

# Step 2: Perform the ADF test to check stationarity
adf_test = adfuller(el_power_series)

print(f"ADF Statistic: {adf_test[0]}")
print(f"p-value: {adf_test[1]}")
print("Critical Values:")
for key, value in adf_test[4].items():
    print(f"   {key}: {value}")

# Step 3: Plot ACF and PACF to identify AR and MA terms
plt.figure(figsize=(10, 5))
plot_pacf(el_power_series, lags=40)
plt.title('Partial Autocorrelation Function (PACF)')
plt.show()

plt.figure(figsize=(10, 5))
plot_acf(el_power_series, lags=40)
plt.title('Autocorrelation Function (ACF)')
plt.show()

# Step 4: Differencing (if necessary) based on ADF test results
d = 1 if adf_test[1] > 0.05 else 0  # Set d=1 for differencing if series is non-stationary
```

```python
# Step 5: Fit ARIMA model (p and q values can be adjusted)
p = 2  # Example value for AR terms
q = 2  # Example value for MA terms

# Fit the ARIMA model
model = ARIMA(el_power_series, order=(p, d, q))
model_fit = model.fit()

# Step 6: Print the summary of the model
print(model_fit.summary())

# Step 7: Calculate AIC and BIC
aic = model_fit.aic
bic = model_fit.bic
print(f"AIC: {aic}")
print(f"BIC: {bic}")

# Step 8: Plot the residuals to check for white noise
residuals = model_fit.resid
plt.figure(figsize=(10, 6))
plt.plot(residuals)
plt.title('Residuals from ARIMA Model')
plt.xlabel('Time')
plt.ylabel('Residuals')
plt.axhline(y=0, color='r', linestyle='--')
plt.grid(True)
plt.show()

# Step 9: Perform diagnostics on the residuals
plt.figure(figsize=(10, 5))
plot_acf(residuals, lags=40)
plt.title('ACF of Residuals')
plt.show()

plt.figure(figsize=(10, 5))
plot_pacf(residuals, lags=40)
plt.title('PACF of Residuals')
plt.show()

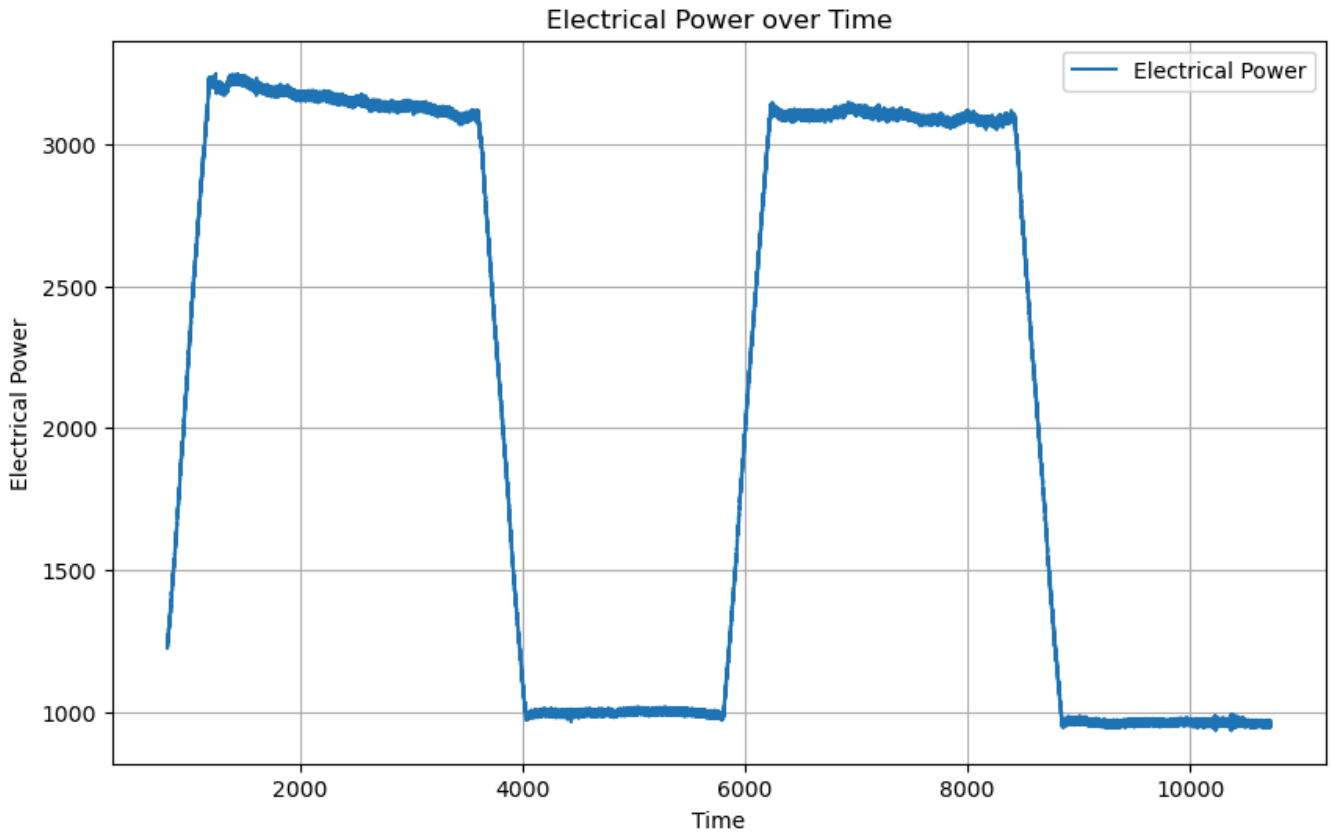
# Step 10: Forecast future values
forecast = model_fit.forecast(steps=10)  # Forecast the next 10 time steps
plt.figure(figsize=(10, 6))
plt.plot(el_power_series.index, el_power_series, label='Historical Data')
plt.plot(np.arange(len(el_power_series), len(el_power_series) + 10), forecast, label='Forecas
plt.title('Forecasted Electrical Power')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.legend()
plt.grid(True)
plt.show()

# Step 11: Calculate RMSE, MAE, and MAPE
# Create a forecast series for the last observed data point
full_forecast = model_fit.get_forecast(steps=10)
predicted_mean = full_forecast.predicted_mean

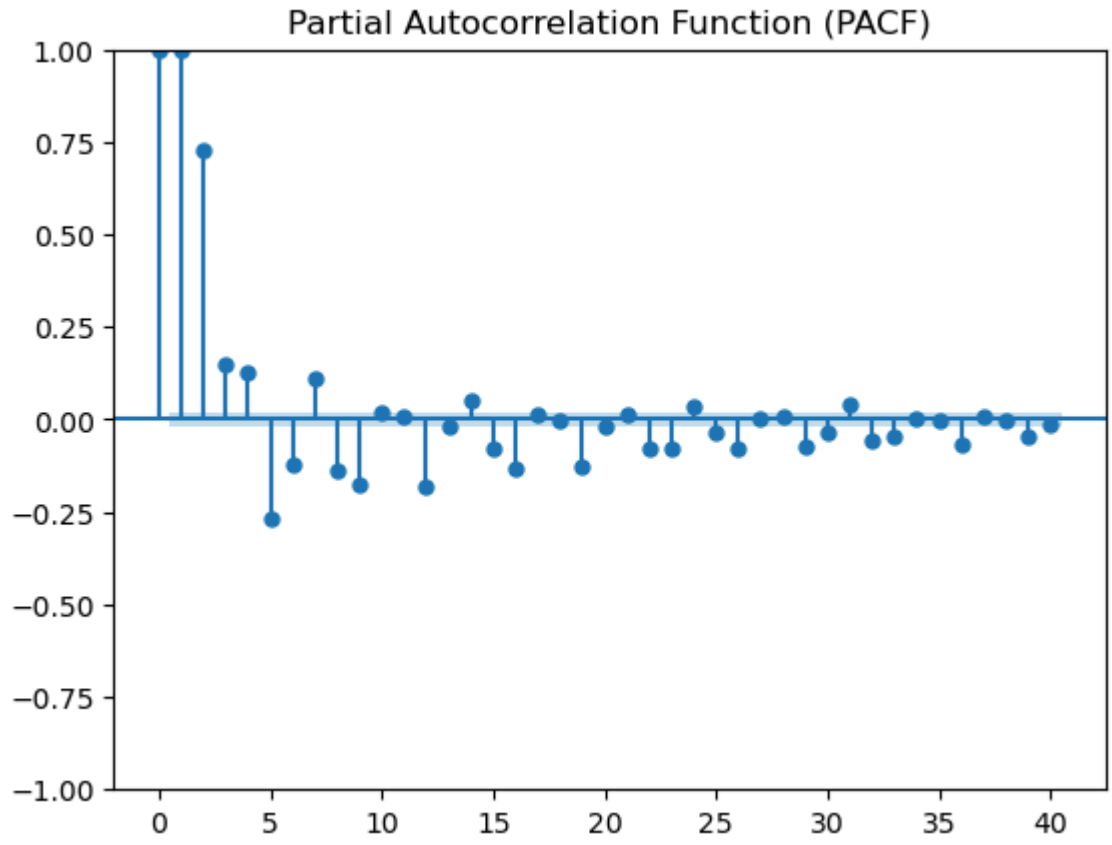
# Extend the series for evaluation
actual_values = el_power_series[-10:].values

# Calculate metrics
rmse = np.sqrt(np.mean((predicted_mean - actual_values) ** 2))
mae = np.mean(np.abs(predicted_mean - actual_values))
mape = np.mean(np.abs((predicted_mean - actual_values) / actual_values)) * 100

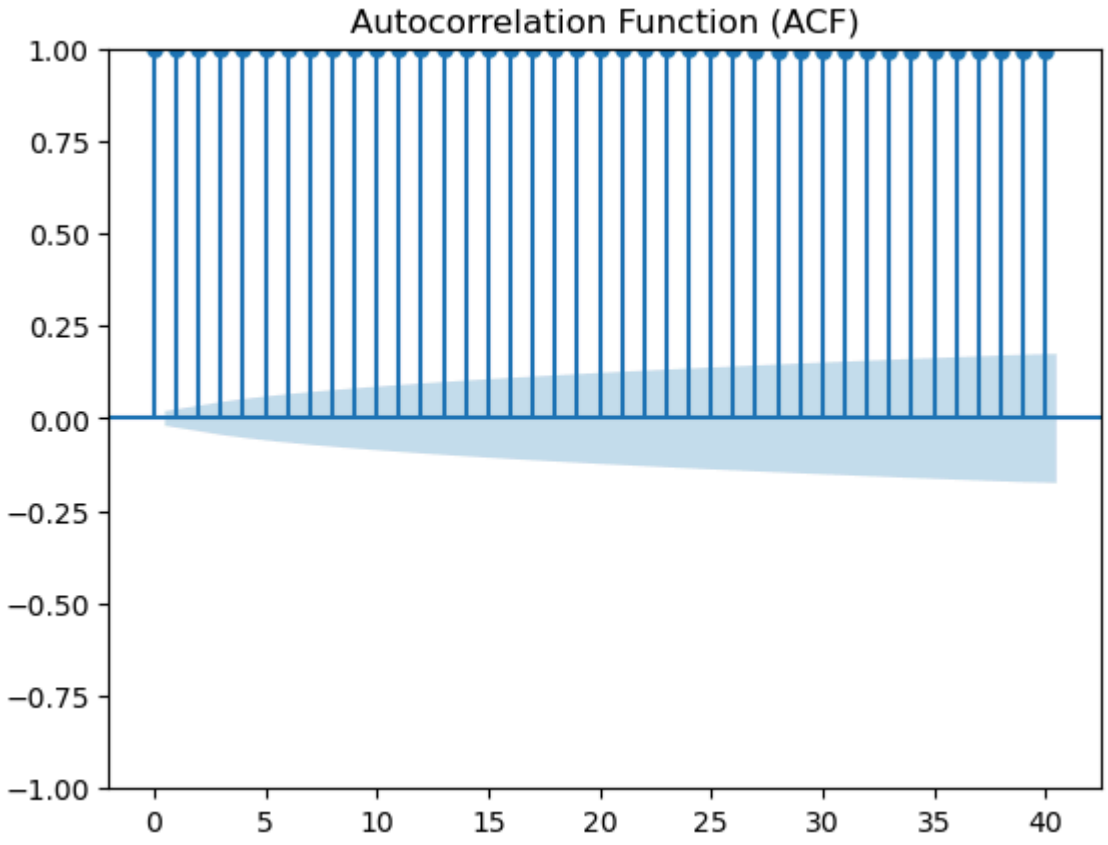
print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
print(f"MAPE: {mape}%")
```




```
ADF Statistic: -2.88003477482922
p-value: 0.04771942827842128
Critical Values:
   1%: -3.4310119774951406
   5%: -2.861832554286939
   10%: -2.566925721514317
<Figure size 1000x500 with 0 Axes>
```




```
<Figure size 1000x500 with 0 Axes>
```



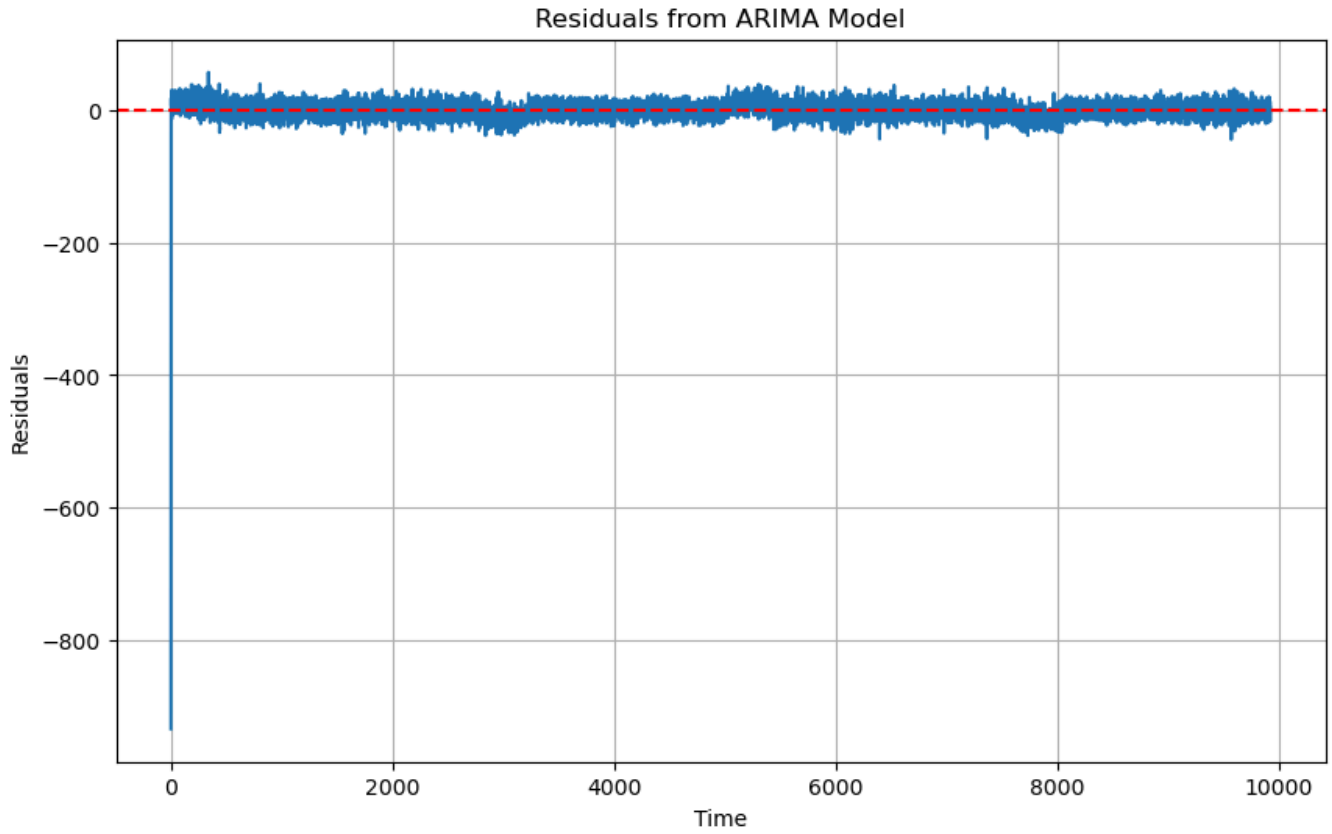
SARIMAX Results

Dep. Variable:	el_power	No. Observations:	9920
Model:	ARIMA(2, 0, 2)	Log Likelihood	-37774.344
Date:	Sat, 05 Oct 2024	AIC	75560.689
Time:	15:26:28	BIC	75603.903
Sample:	0	HQIC	75575.322
	- 9920		
Covariance Type:	opg		

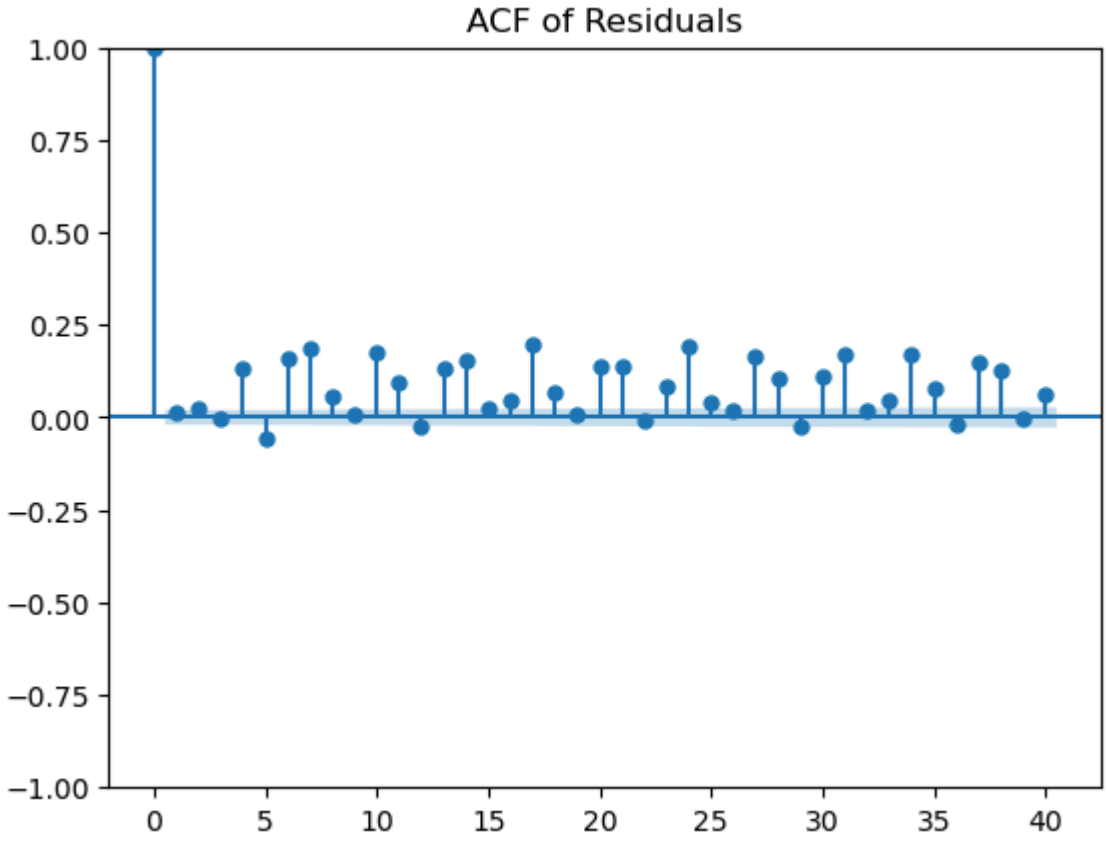
	coef	std err	z	P>|z|	[0.025	0.975]
const	2162.3684	474.199	4.560	0.000	1232.956	3091.781
ar.L1	0.7264	0.016	45.961	0.000	0.695	0.757
ar.L2	0.2735	0.016	17.306	0.000	0.243	0.304
ma.L1	-0.9386	0.012	-79.238	0.000	-0.962	-0.915
ma.L2	0.5381	0.010	52.609	0.000	0.518	0.558
sigma2	118.1314	1.791	65.970	0.000	114.622	121.641

Ljung-Box (L1) (Q):	4.53	Jarque-Bera (JB):	43.22
Prob(Q):	0.03	Prob(JB):	0.00
Heteroskedasticity (H):	0.80	Skew:	0.04
Prob(H) (two-sided):	0.00	Kurtosis:	3.31

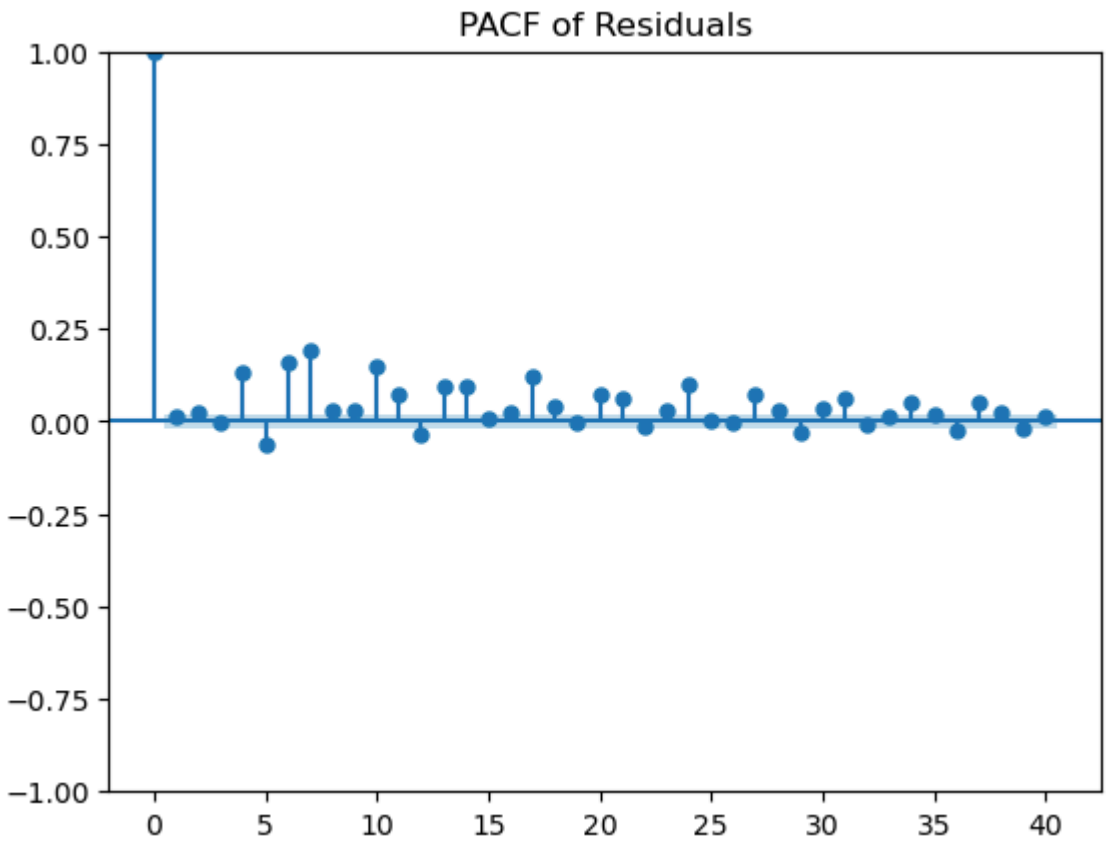
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
AIC: 75560.6889696246
BIC: 75603.90281882627

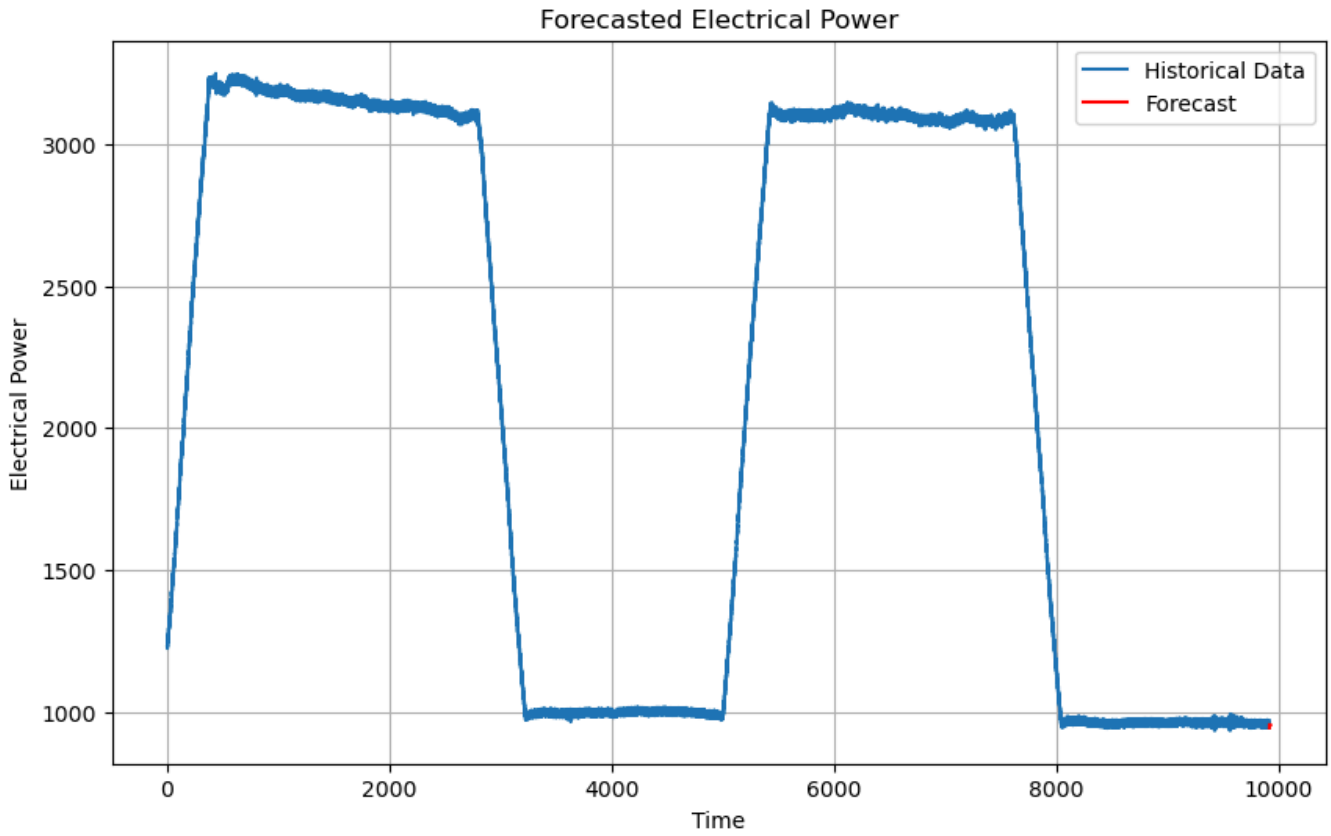


<Figure size 1000x500 with 0 Axes>



<Figure size 1000x500 with 0 Axes>





RMSE: 9.533227610716276
MAE: 8.435868927015713
MAPE: 0.8798867618262285%

For ARIMA Model

AIC: 75560.6889696246

BIC: 75603.90281882627

RMSE: 9.533227610716276

MAE: 8.435868927015713

MAPE: 0.8798867618262285 %

5. SEASONAL AUTOREGRESSIVE INTEGRATED MOVING AVERAGE (SARIMA)

To modify the code for SARIMA (Seasonal ARIMA), we need to incorporate seasonality into the model.

SARIMA extends ARIMA by including seasonal components. The SARIMA model is represented as:

SARIMA(p, d, q)(P, D, Q, m), where:

p, d, q: Same as ARIMA, referring to the non-seasonal autoregressive, differencing, and moving average terms.

P, D, Q: These represent the seasonal autoregressive, seasonal differencing, and seasonal moving average terms.

m: The number of time steps per seasonal cycle (for example, 12 for monthly data if the seasonality is yearly).

We'll need to determine the seasonal period (m) based on the data, and then configure the model to handle both seasonal and non-seasonal components.

Steps for SARIMA Modification:

Check for Seasonality: First, we analyze the data to check for any seasonal patterns (using ACF, PACF, or domain knowledge to determine the seasonal period m).

Fit SARIMA Model: We fit a SARIMA model using the identified seasonal and non-seasonal parameters.

Analyze Residuals and Forecast: Similar to ARIMA, we'll check residuals and perform forecasting.

The code follows a typical workflow for time series analysis and forecasting using SARIMA.

Steps involved: -

1. data loading
2. exploration
3. stationarity testing
4. model fitting
5. diagnostics
6. forecasting

In [5]:
```
# Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_pacf, plot_acf
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
import warnings
```

```python
warnings.filterwarnings('ignore')

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Extract the relevant time series (el_power column)
el_power_series = data['el_power']

# Step 1: Plot the raw data to visualize the time series
plt.figure(figsize=(10, 6))
plt.plot(data['time'], el_power_series, label='Electrical Power')
plt.title('Electrical Power over Time')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.grid(True)
plt.legend()
plt.show()

# Step 2: Perform the ADF test to check stationarity
adf_test = adfuller(el_power_series)
print(f"ADF Statistic: {adf_test[0]}")
print(f"p-value: {adf_test[1]}")
print("Critical Values:")
for key, value in adf_test[4].items():
    print(f"   {key}: {value}")

# Step 3: Plot ACF and PACF to identify AR and MA terms
plt.figure(figsize=(10, 5))
plot_pacf(el_power_series, lags=40)
plt.title('Partial Autocorrelation Function (PACF)')
plt.show()

plt.figure(figsize=(10, 5))
plot_acf(el_power_series, lags=40)
plt.title('Autocorrelation Function (ACF)')
plt.show()

# Step 4: Differencing (if necessary) based on ADF test results
d = 1 if adf_test[1] > 0.05 else 0

# Step 5: Identify the seasonal period (m)
m = 24  # Adjust this based on your data's frequency

# Step 6: Fit SARIMA model to the 'el_power' time series data
p, q = 2, 2
P, Q, D = 1, 1, 1

model = SARIMAX(el_power_series, order=(p, d, q), seasonal_order=(P, D, Q, m))
model_fit = model.fit()

# Step 7: Summary of the SARIMA model
print(model_fit.summary())

# Step 8: Plot the residuals to check for randomness
plt.figure(figsize=(10, 5))
plt.plot(model_fit.resid)
plt.title('Residuals of the SARIMA Model')
plt.grid(True)
plt.show()

# Plot ACF and PACF of residuals to check for autocorrelation
plot_acf(model_fit.resid, lags=40)
plt.title('ACF of Residuals')
plt.show()

plot_pacf(model_fit.resid, lags=40)
plt.title('PACF of Residuals')
plt.show()
```

```python
# Step 9: Ljung-Box test for residual autocorrelation
from statsmodels.stats.diagnostic import acorr_ljungbox
lb_test = acorr_ljungbox(model_fit.resid, lags=[10], return_df=True)
print("Ljung-Box Test Results:")
print(lb_test)

# Step 10: Forecasting the next 10 points using the fitted SARIMA model
forecast = model_fit.get_forecast(steps=10)
forecast_ci = forecast.conf_int()

# Plot the forecasted values
plt.figure(figsize=(10, 6))
plt.plot(data['time'], el_power_series, label='Observed Data')
plt.plot(np.arange(data['time'].iloc[-1], data['time'].iloc[-1] + 10), forecast.predicted_mea
plt.fill_between(np.arange(data['time'].iloc[-1], data['time'].iloc[-1] + 10),
                 forecast_ci.iloc[:, 0], forecast_ci.iloc[:, 1], color='gray', alpha=0.3)
plt.title('SARIMA Forecast of Electrical Power')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.legend()
plt.grid(True)
plt.show()

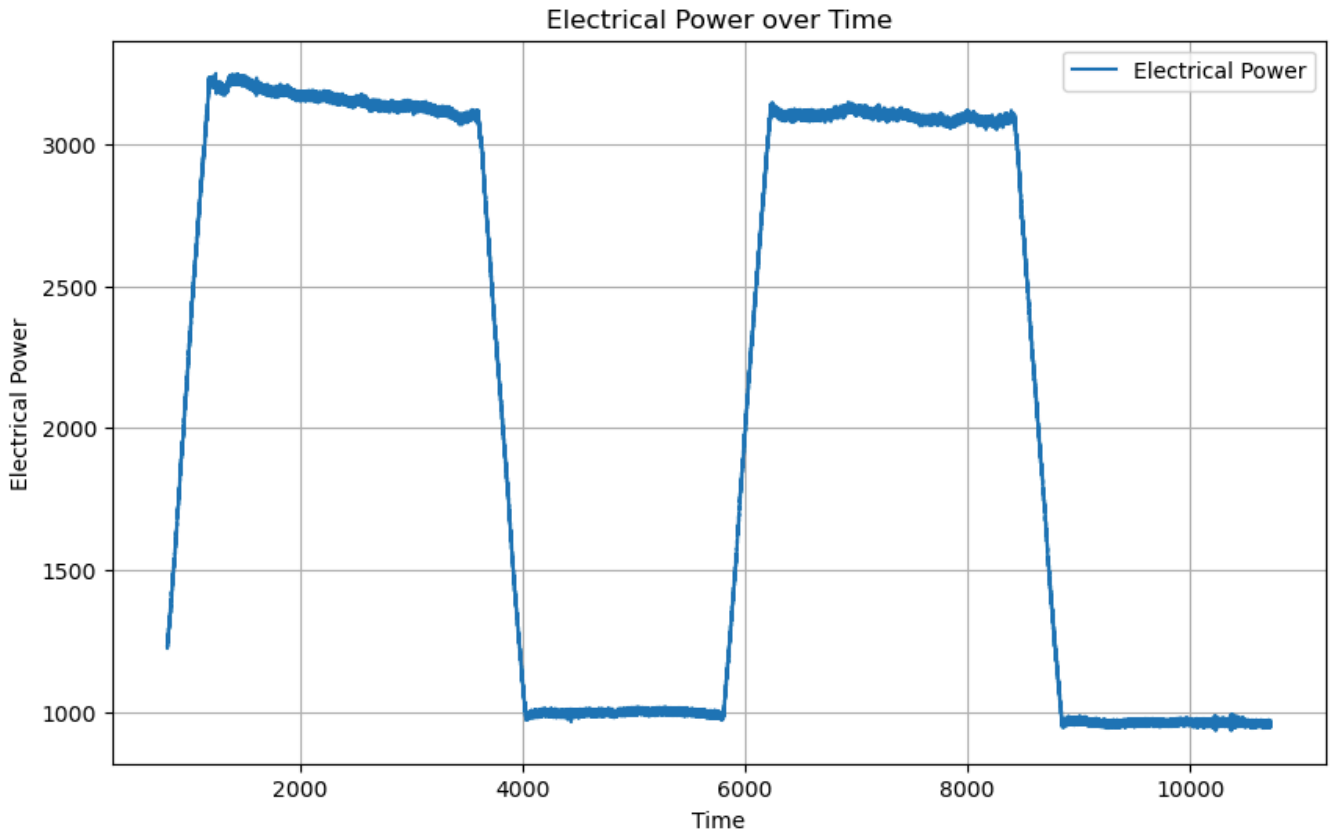
# Step 11: Calculate AIC and BIC
aic = model_fit.aic
bic = model_fit.bic
print(f'AIC: {aic}')
print(f'BIC: {bic}')

# Step 12: Calculate RMSE, MAE, and MAPE
# Get the predicted values from the model
predicted_values = model_fit.predict(start=0, end=len(el_power_series)-1)

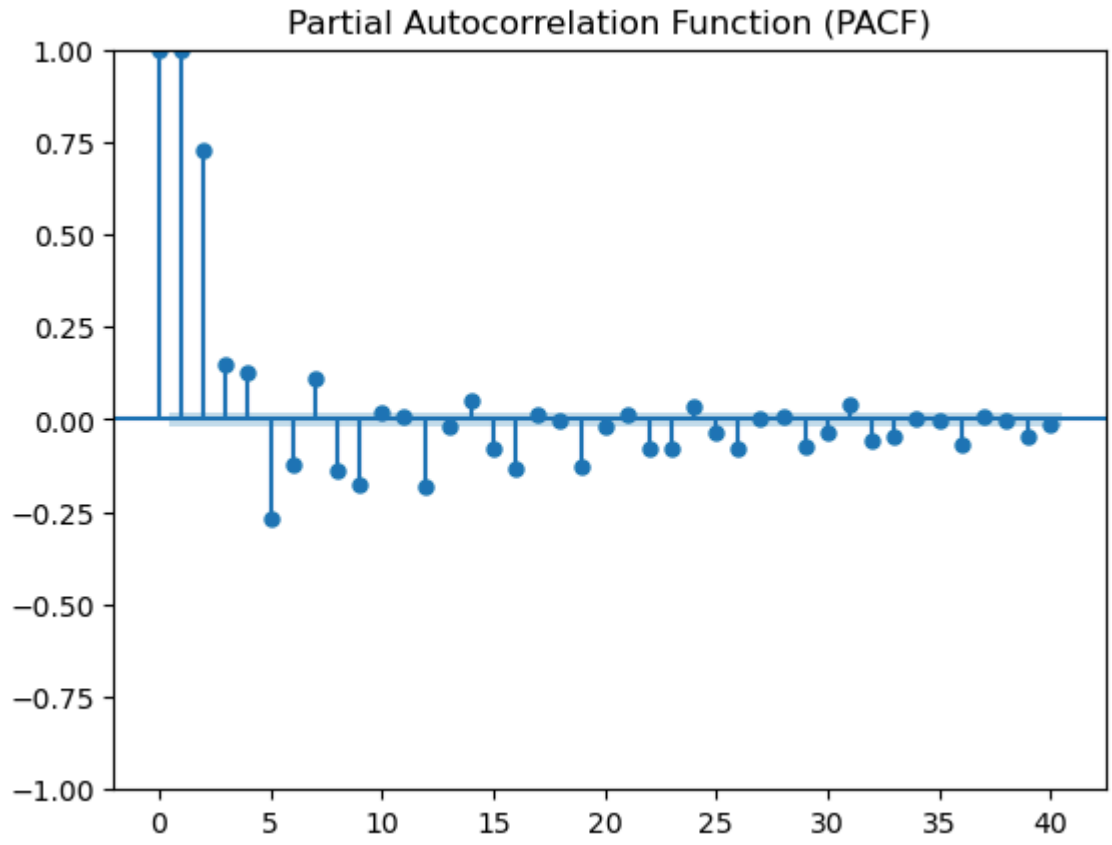
# Calculate RMSE
rmse = np.sqrt(np.mean((predicted_values - el_power_series) ** 2))
print(f'RMSE: {rmse}')

# Calculate MAE
mae = np.mean(np.abs(predicted_values - el_power_series))
print(f'MAE: {mae}')

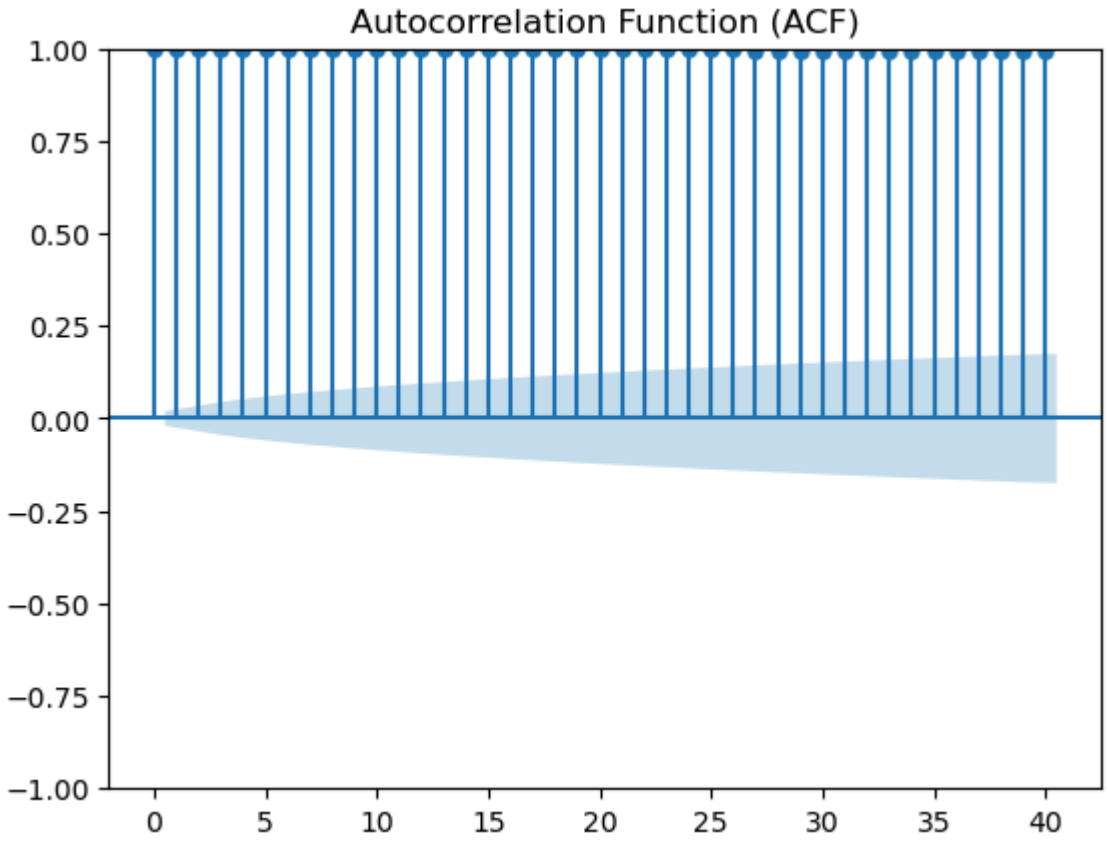
# Calculate MAPE
mape = np.mean(np.abs((predicted_values - el_power_series) / el_power_series)) * 100
print(f'MAPE: {mape:.2f}%')
```




```
ADF Statistic: -2.88003477482922
p-value: 0.04771942827842128
Critical Values:
   1%: -3.4310119774951406
   5%: -2.861832554286939
   10%: -2.566925721514317
<Figure size 1000x500 with 0 Axes>
```




```
<Figure size 1000x500 with 0 Axes>
```



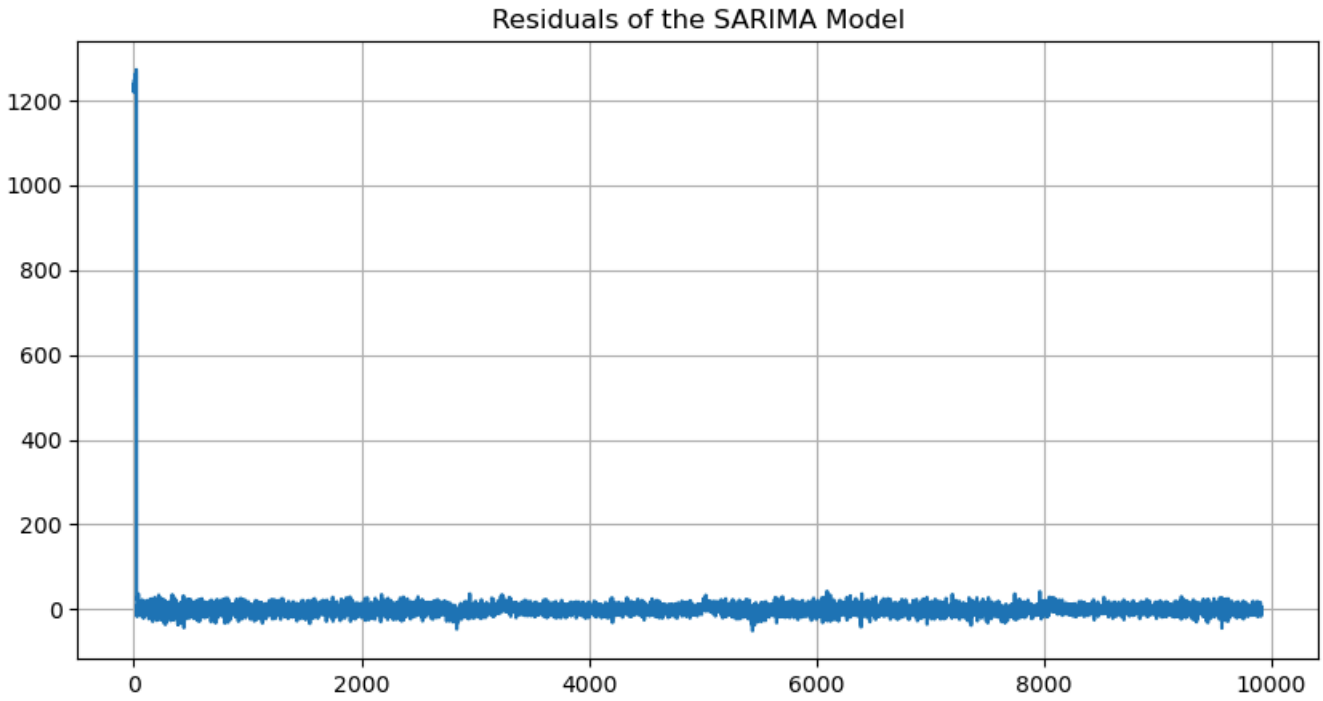
SARIMAX Results

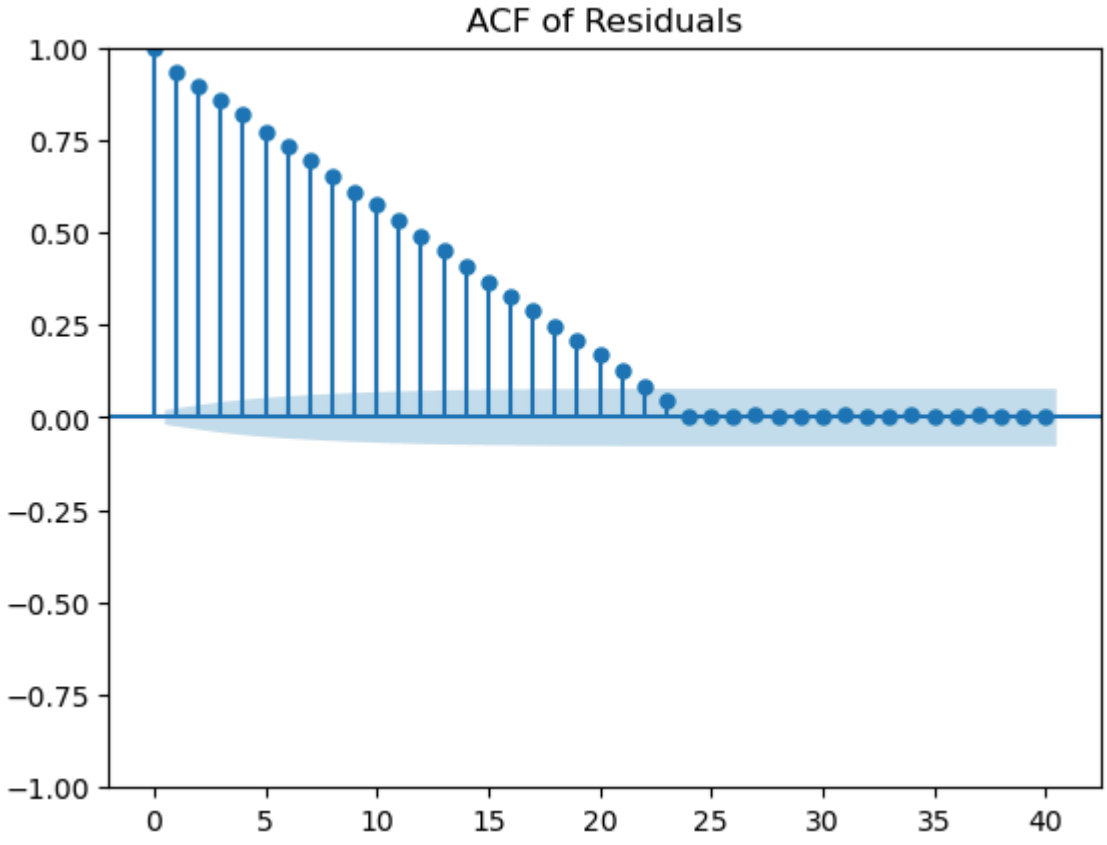
Dep. Variable:	el_power	No. Observations:	9920
Model:	SARIMAX(2, 0, 2)x(1, 1, [1], 24)	Log Likelihood	-36630.819
Date:	Sat, 05 Oct 2024	AIC	73275.638
Time:	15:28:28	BIC	73326.037
Sample:	0	HQIC	73292.706
	- 9920		
Covariance Type:	opg		

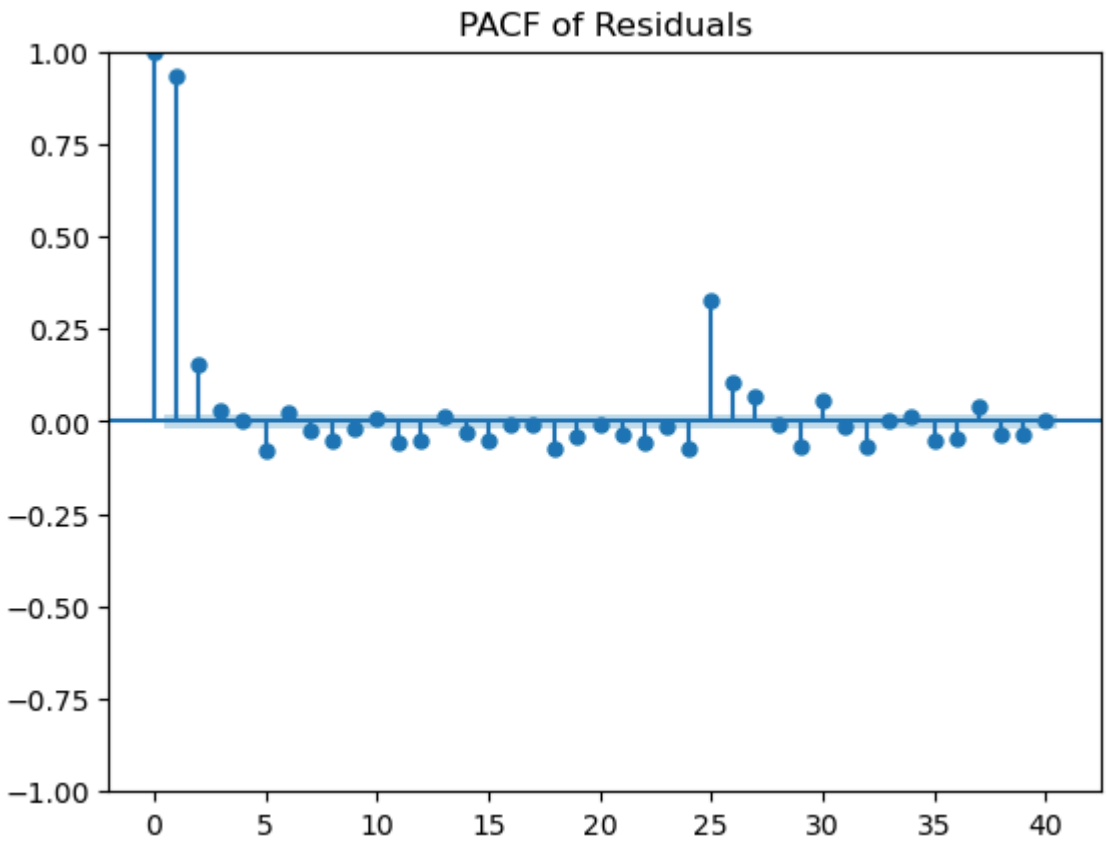
	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.3878	0.012	32.484	0.000	0.364	0.411
ar.L2	0.6118	0.012	51.302	0.000	0.588	0.635
ma.L1	-0.7682	0.013	-57.709	0.000	-0.794	-0.742
ma.L2	0.2108	0.012	16.963	0.000	0.186	0.235
ar.S.L24	0.0380	0.016	2.378	0.017	0.007	0.069
ma.S.L24	-0.6270	0.013	-48.090	0.000	-0.653	-0.601
sigma2	95.9606	1.266	75.799	0.000	93.479	98.442

Ljung-Box (L1) (Q):	2.99	Jarque-Bera (JB):	257.99
Prob(Q):	0.08	Prob(JB):	0.00
Heteroskedasticity (H):	0.84	Skew:	-0.11
Prob(H) (two-sided):	0.00	Kurtosis:	3.76

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

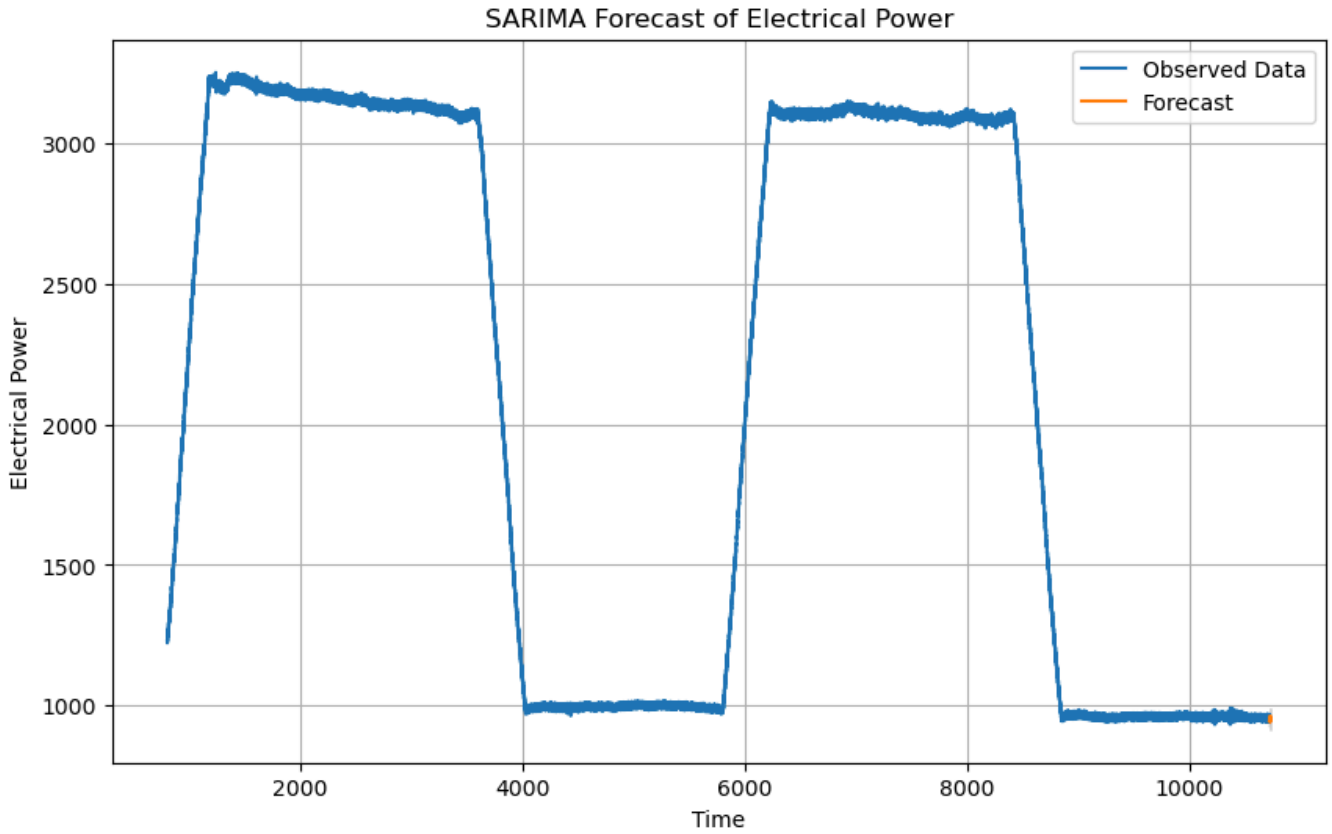

Residuals of the SARIMA Model
1200
1000
800
600
400
200
0
0
2000
4000
6000
8000
10000


ACF of Residuals
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30
35
40



Ljung-Box Test Results:

	lb_stat	lb_pvalue
10	57831.72747	0.0



AIC: 73275.63765354396
BIC: 73326.03685494247
RMSE: 61.978323577212834
MAE: 10.56218382890263
MAPE: 0.68%

For SARIMA Model

AIC: 73275.63765354396

BIC: 73326.03685494247

RMSE: 61.978323577212834

MAE: 10.56218382890263

MAPE: 0.68 %

6. ARCH (Autoregressive Conditional Heteroskedasticity) model

In [6]:
```
# Import required libraries
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from arch import arch_model  # Import the ARCH model
import warnings

warnings.filterwarnings('ignore')

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Extract the relevant time series (el_power column)
el_power_series = data['el_power']


# Step 1: Plot the raw data to visualize the time series
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Electrical Power')
plt.title('Electrical Power over Time')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.grid(True)
plt.legend()
plt.show()

# Step 2: Perform the ADF test to check stationarity
adf_test = adfuller(el_power_series)

# Step 3: Fit ARIMA model to obtain residuals for ARCH model and calculate AIC, BIC, RMSE, MA
p, d, q = 1, 0, 0  # Example ARIMA(1, 0, 0) model for residuals

# Fit ARIMA model
arima_model = ARIMA(el_power_series, order=(p, d, q))
model_fit = arima_model.fit()

# Get residuals from ARIMA model
residuals = model_fit.resid

# Calculate AIC and BIC
aic = model_fit.aic
bic = model_fit.bic
print(f"AIC: {aic}")
print(f"BIC: {bic}")

# Calculate RMSE, MAE, and MAPE
# Predicted values (fitted values) from the ARIMA model
predictions = model_fit.fittedvalues
```

```python
# RMSE: Root Mean Squared Error
rmse = np.sqrt(mean_squared_error(el_power_series, predictions))
print(f"RMSE: {rmse}")

# MAE: Mean Absolute Error
mae = mean_absolute_error(el_power_series, predictions)
print(f"MAE: {mae}")

# MAPE: Mean Absolute Percentage Error
mape = np.mean(np.abs((el_power_series - predictions) / el_power_series)) * 100
print(f"MAPE: {mape:.2f}%")

# Step 4: Fit ARCH model to the residuals
arch_model_instance = arch_model(residuals, vol='ARCH', p=1)  # ARCH(1) model
arch_fit = arch_model_instance.fit()

# Step 5: Summary of the ARCH model
print(arch_fit.summary())

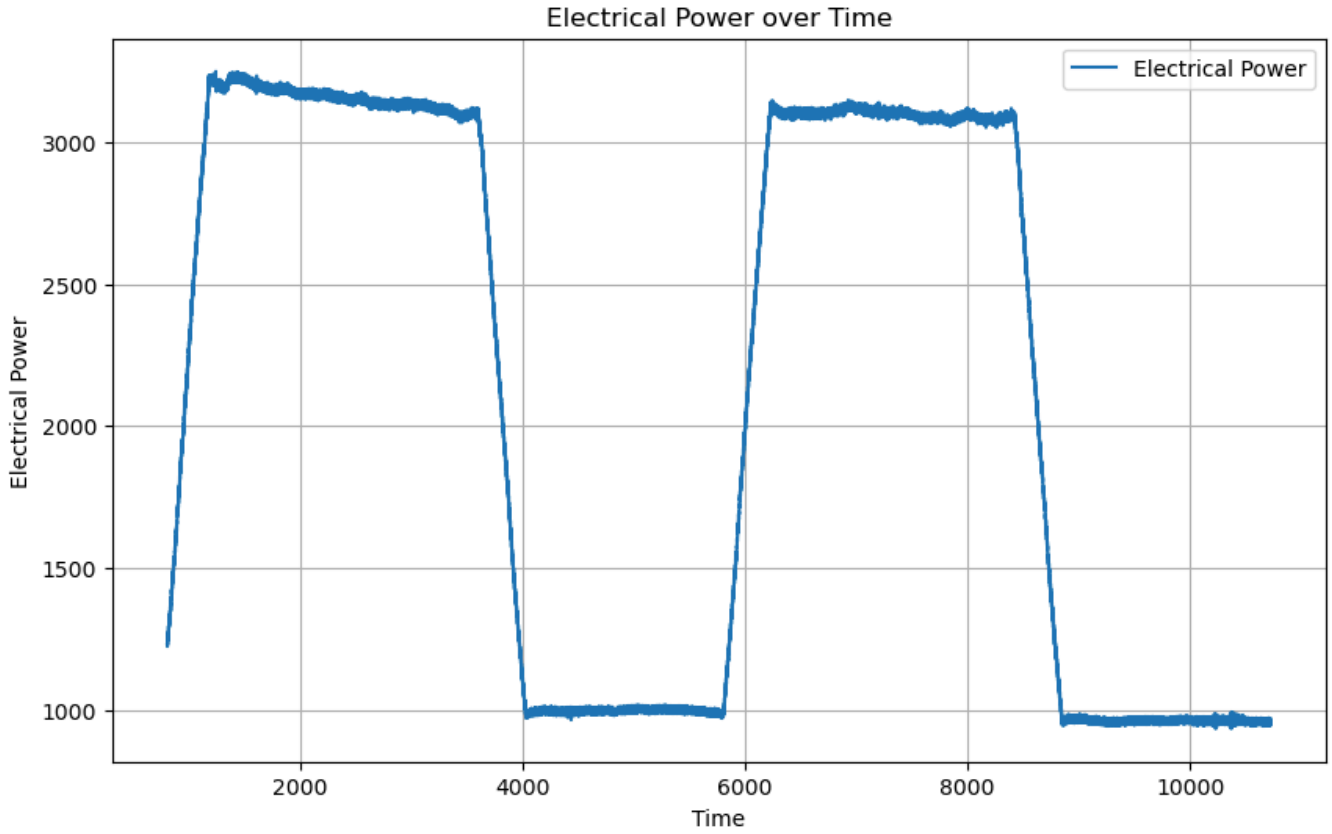
# Step 6: Plot the residuals to check for randomness
plt.figure(figsize=(10, 5))
plt.plot(residuals)
plt.title('Residuals of the ARIMA Model')
plt.grid(True)
plt.show()

# Step 7: Plot ACF and PACF of residuals to check for autocorrelation
plot_acf(residuals, lags=40)
plt.title('ACF of Residuals')
plt.show()

plot_pacf(residuals, lags=40)
plt.title('PACF of Residuals')
plt.show()

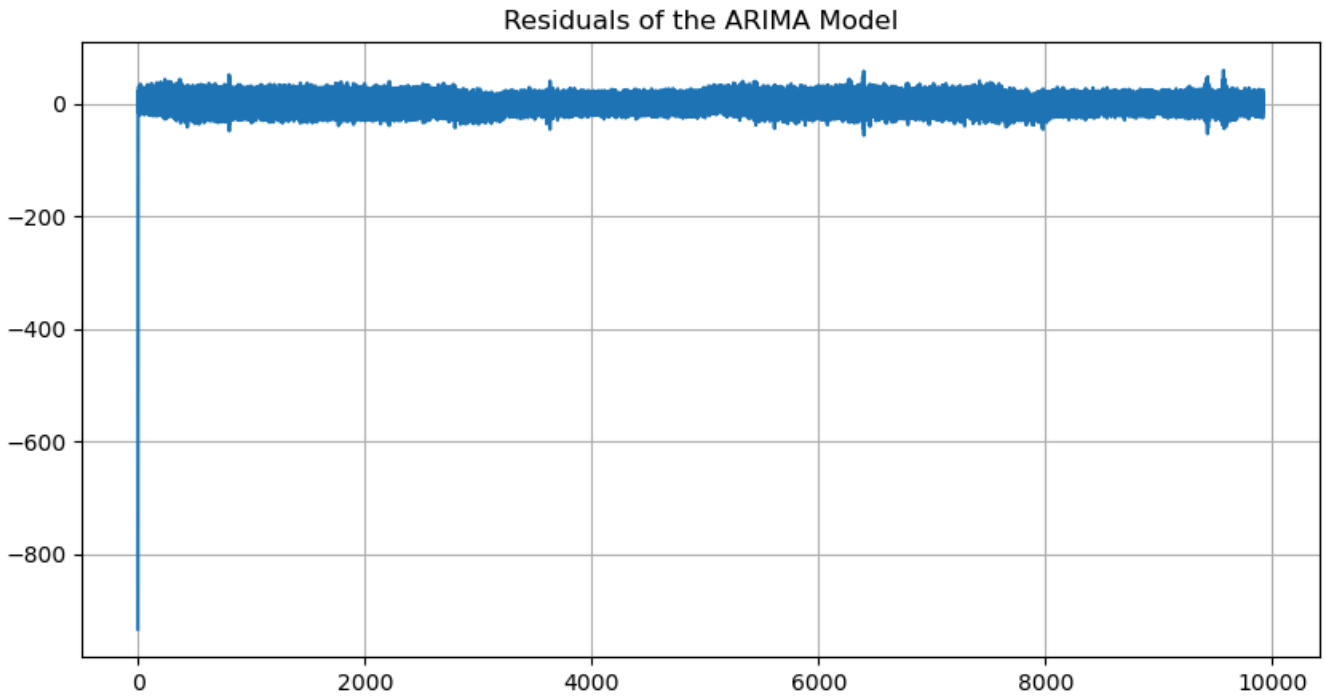
# Step 8: Forecasting volatility using the ARCH model
arch_forecast = arch_fit.forecast(horizon=10)
forecast_mean = arch_forecast.mean.iloc[-1]
forecast_variance = arch_forecast.variance.iloc[-1]

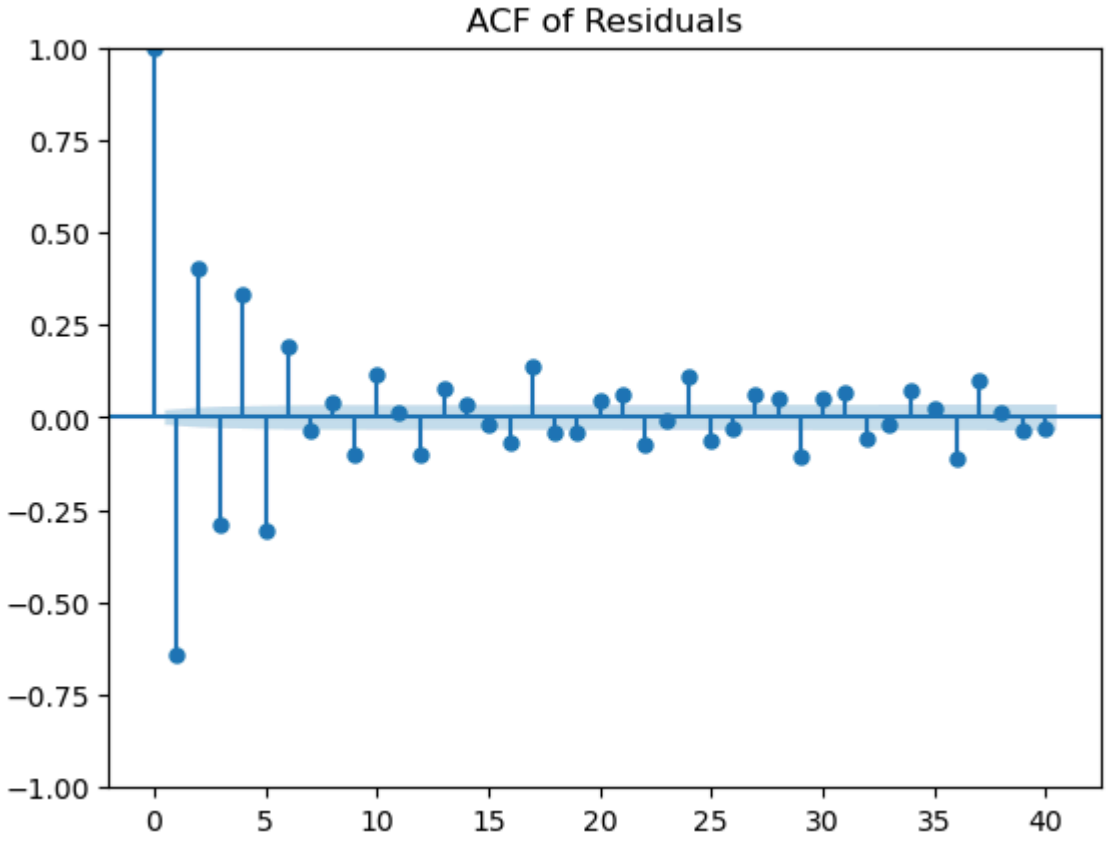
# Step 9: Plotting the forecasted values
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Observed Data')
plt.plot(np.arange(len(data), len(data) + 10), forecast_mean, label='Forecast Mean')
plt.fill_between(np.arange(len(data), len(data) + 10),
                 forecast_variance, color='gray', alpha=0.3, label='Forecast Variance')
plt.title('Forecast of Electrical Power (with ARCH Model)')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.legend()
plt.grid(True)
plt.show()
```

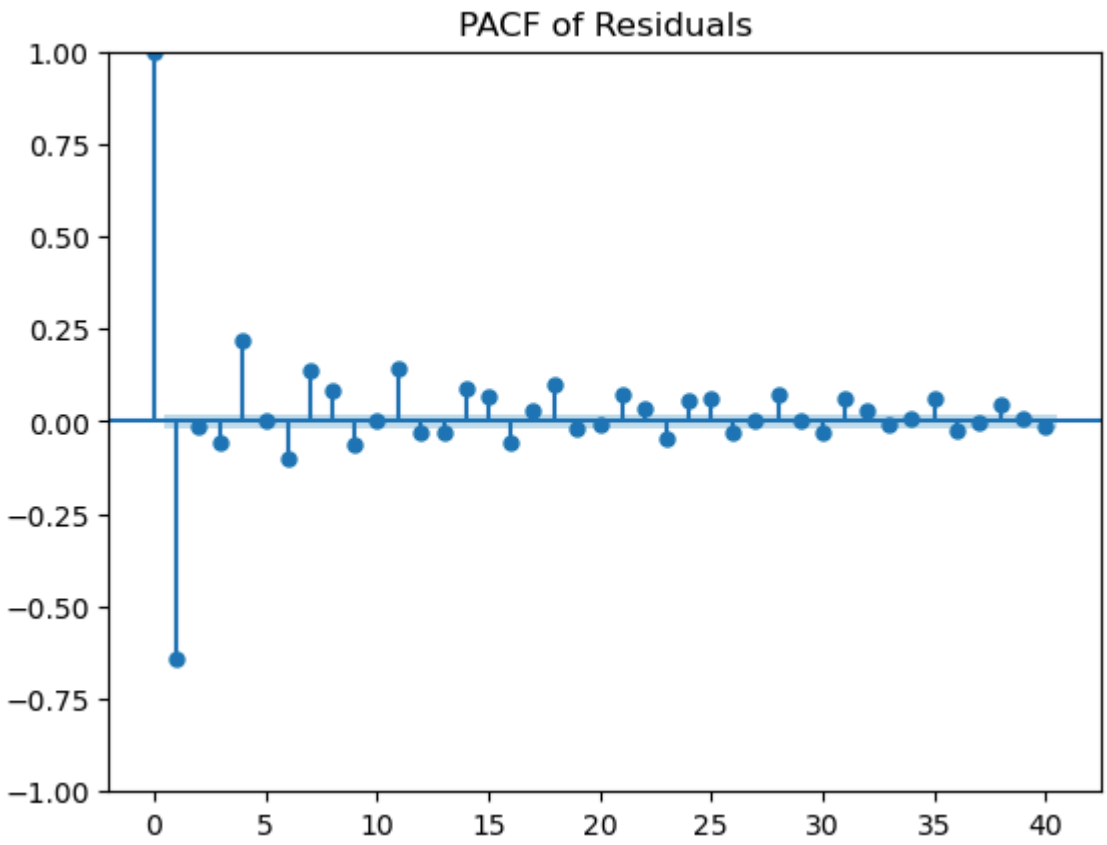

Electrical Power over Time
Electrical Power
3000
2500
2000
1500
1000
Electrical Power
2000
4000
6000
8000
10000
Time

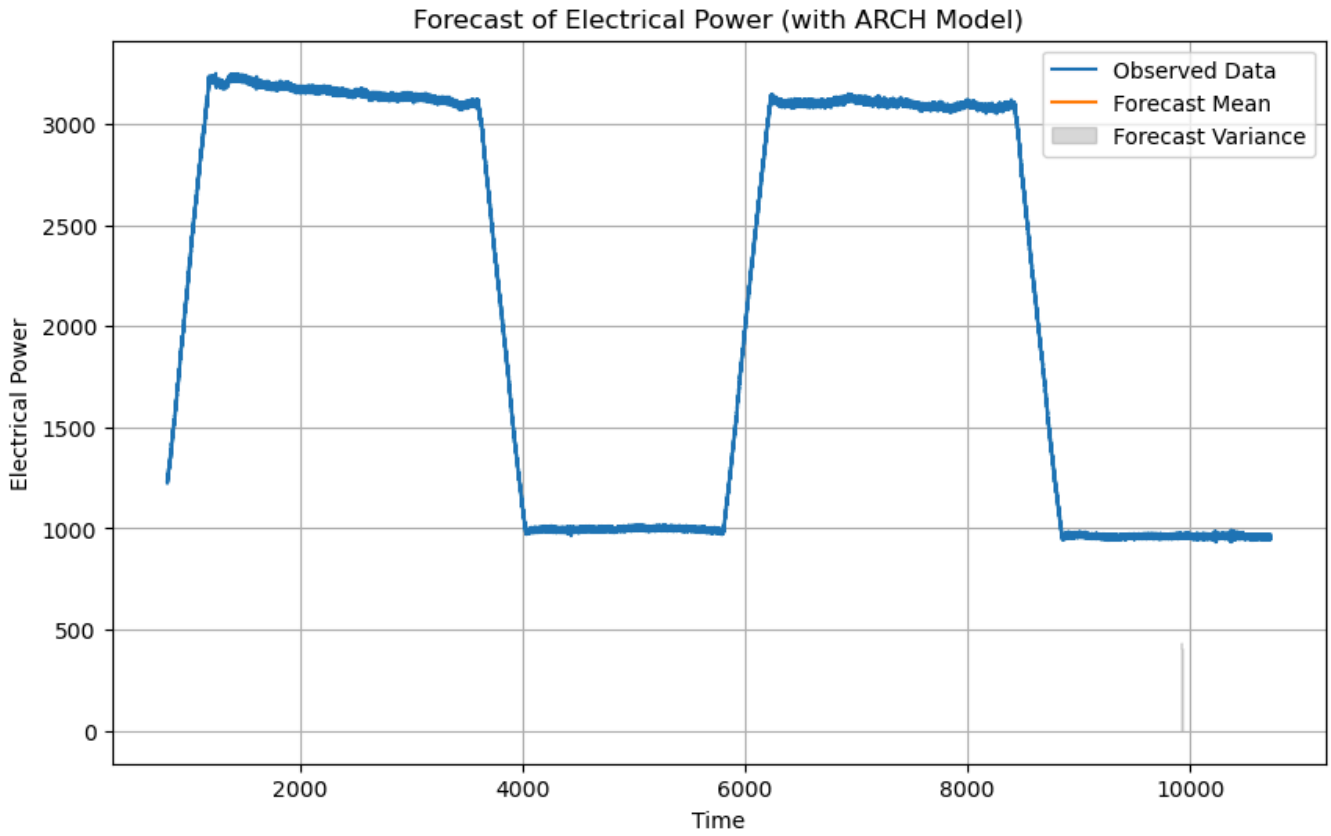
```
AIC: 87581.4989274149
BIC: 87603.10585201574
RMSE: 22.069273924782024
MAE: 18.65331198633499
MAPE: 1.06%
Iteration:      1,   Func. Count:      5,   Neg. LLF: 45150.37268186996
Iteration:      2,   Func. Count:     10,   Neg. LLF: 45050.42886118581
Iteration:      3,   Func. Count:     15,   Neg. LLF: 45012.87774301472
Iteration:      4,   Func. Count:     20,   Neg. LLF: 43833.46755488436
Iteration:      5,   Func. Count:     24,   Neg. LLF: 43830.86382790003
Iteration:      6,   Func. Count:     28,   Neg. LLF: 43829.98541096438
Iteration:      7,   Func. Count:     32,   Neg. LLF: 43825.72273771687
Iteration:      8,   Func. Count:     36,   Neg. LLF: 43820.96753396164
Iteration:      9,   Func. Count:     40,   Neg. LLF: 43820.879164542945
Iteration:     10,   Func. Count:     44,   Neg. LLF: 43820.8757398615
Iteration:     11,   Func. Count:     48,   Neg. LLF: 43820.87558635224
Iteration:     12,   Func. Count:     52,   Neg. LLF: 43820.87558744811
Iteration:     13,   Func. Count:     57,   Neg. LLF: 43820.8764610931
Optimization terminated successfully    (Exit mode 0)
            Current function value: 43820.87558460212
            Iterations: 13
            Function evaluations: 67
            Gradient evaluations: 13
                     Constant Mean - ARCH Model Results
==============================================================================
Dep. Variable:                    None   R-squared:                        0.000
Mean Model:             Constant Mean   Adj. R-squared:                   0.000
Vol Model:                       ARCH   Log-Likelihood:               -43820.9
Distribution:                  Normal   AIC:                           87647.8
Method:            Maximum Likelihood   BIC:                           87669.4
                                        No. Observations:                 9920
Date:                Sat, Oct 05 2024   Df Residuals:                     9919
Time:                        15:28:32   Df Model:                            1
                            Mean Model
============================================================================
                 coef    std err          t      P>|t|  95.0% Conf. Int.
----------------------------------------------------------------------------
mu            -0.0303      0.134     -0.226      0.821 [ -0.293,  0.232]
                           Volatility Model
============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
----------------------------------------------------------------------------
omega        332.3738     13.621     24.402 1.638e-131 [3.057e+02,3.591e+02]
alpha[1]       0.1816  4.568e-02      3.976  7.001e-05   [9.211e-02,  0.271]
============================================================================

Covariance estimator: robust
```


Residuals of the ARIMA Model
0
−200
−400
−600
−800
0
2000
4000
6000
8000
10000


ACF of Residuals
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30
35
40





For ARCH Model

AIC: 87581.4989274149

BIC: 87603.10585201574

RMSE: 22.069273924782024

MAE: 18.65331198633499

MAPE: 1.06%

7. GARCH (Generalized Autoregressive Conditional Heteroskedasticity)

In [7]:
```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from arch import arch_model  # Import the GARCH model
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from sklearn.metrics import mean_squared_error, mean_absolute_error  # Import evaluation metr
import warnings

warnings.filterwarnings('ignore')

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Extract the relevant time series (el_power column)
el_power_series = data['el_power']

# Step 1: Plot the raw data to visualize the time series
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Electrical Power')
plt.title('Electrical Power over Time')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.grid(True)
plt.legend()
plt.show()

# Step 2: Perform the ADF test to check stationarity
adf_test = adfuller(el_power_series)

print(f"ADF Statistic: {adf_test[0]}")
print(f"p-value: {adf_test[1]}")
print("Critical Values:")
for key, value in adf_test[4].items():
    print(f"   {key}: {value}")

# Step 3: Fit ARIMA model to obtain residuals for GARCH model
p, d, q = 1, 0, 0  # Example ARIMA(1, 0, 0) model for residuals

# Fit ARIMA model
arima_model = ARIMA(el_power_series, order=(p, d, q))
model_fit = arima_model.fit()

# Step 3.1: Calculate and Print AIC and BIC
print(f"\nARIMA Model AIC: {model_fit.aic}")
print(f"ARIMA Model BIC: {model_fit.bic}")

# Step 3.2: Calculate RMSE, MAE, and MAPE
# Since ARIMA is fitted on the entire dataset, we'll use in-sample predictions
predictions = model_fit.fittedvalues
actual = el_power_series

# Calculate metrics
rmse = np.sqrt(mean_squared_error(actual, predictions))
mae = mean_absolute_error(actual, predictions)
mape = np.mean(np.abs((actual - predictions) / actual)) * 100

print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
```

```
print(f"MAPE: {mape}%")

# Step 4: Get residuals from ARIMA model
residuals = model_fit.resid

# Step 5: Fit GARCH model to the residuals obtained from the ARIMA model.
garch_model = arch_model(residuals, vol='Garch', p=1, q=1)  # GARCH(1, 1) model
garch_fit = garch_model.fit(disp='off')

# Step 6: Summary of the GARCH model
print("\nGARCH Model Summary:")
print(garch_fit.summary())

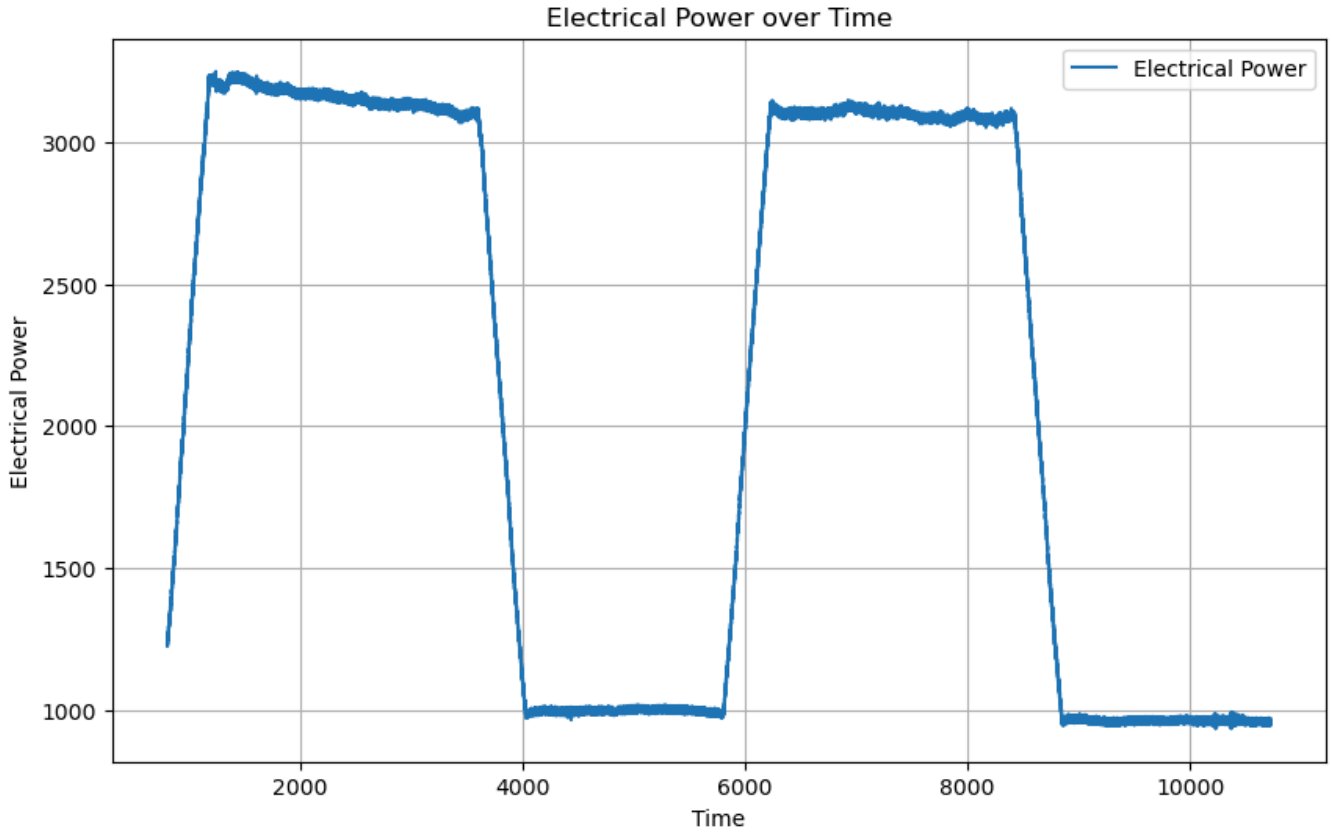
# Step 7: Plot the residuals to check for randomness
plt.figure(figsize=(10, 5))
plt.plot(residuals)
plt.title('Residuals of the ARIMA Model')
plt.grid(True)
plt.show()

# Step 8: Plot ACF and PACF of residuals to check for autocorrelation
plot_acf(residuals, lags=40)
plt.title('ACF of Residuals')
plt.show()

plot_pacf(residuals, lags=40)
plt.title('PACF of Residuals')
plt.show()

# Step 9: Forecasting volatility using the GARCH model
garch_forecast = garch_fit.forecast(horizon=10)
forecast_mean = garch_forecast.mean.iloc[-1]
forecast_variance = garch_forecast.variance.iloc[-1]

# Step 10: Plotting the forecasted values
plt.figure(figsize=(10, 6))
plt.plot(data['time'], data['el_power'], label='Observed Data')
forecast_time = np.arange(len(data), len(data) + 10)
plt.plot(forecast_time, forecast_mean, label='Forecast Mean', marker='o')
plt.fill_between(forecast_time,
                 forecast_mean - np.sqrt(forecast_variance),
                 forecast_mean + np.sqrt(forecast_variance),
                 color='gray', alpha=0.3, label='Forecast Variance')
plt.title('Forecast of Electrical Power (with GARCH Model)')
plt.xlabel('Time')
plt.ylabel('Electrical Power')
plt.legend()
plt.grid(True)
plt.show()
```

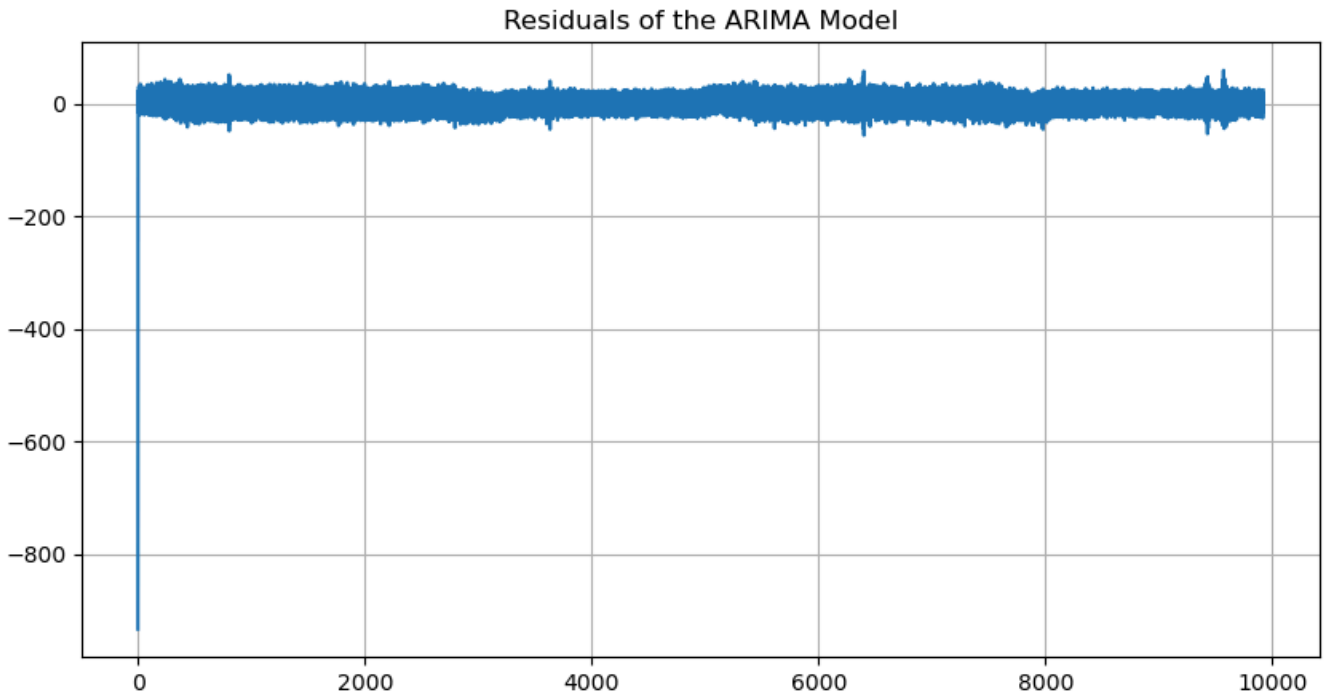


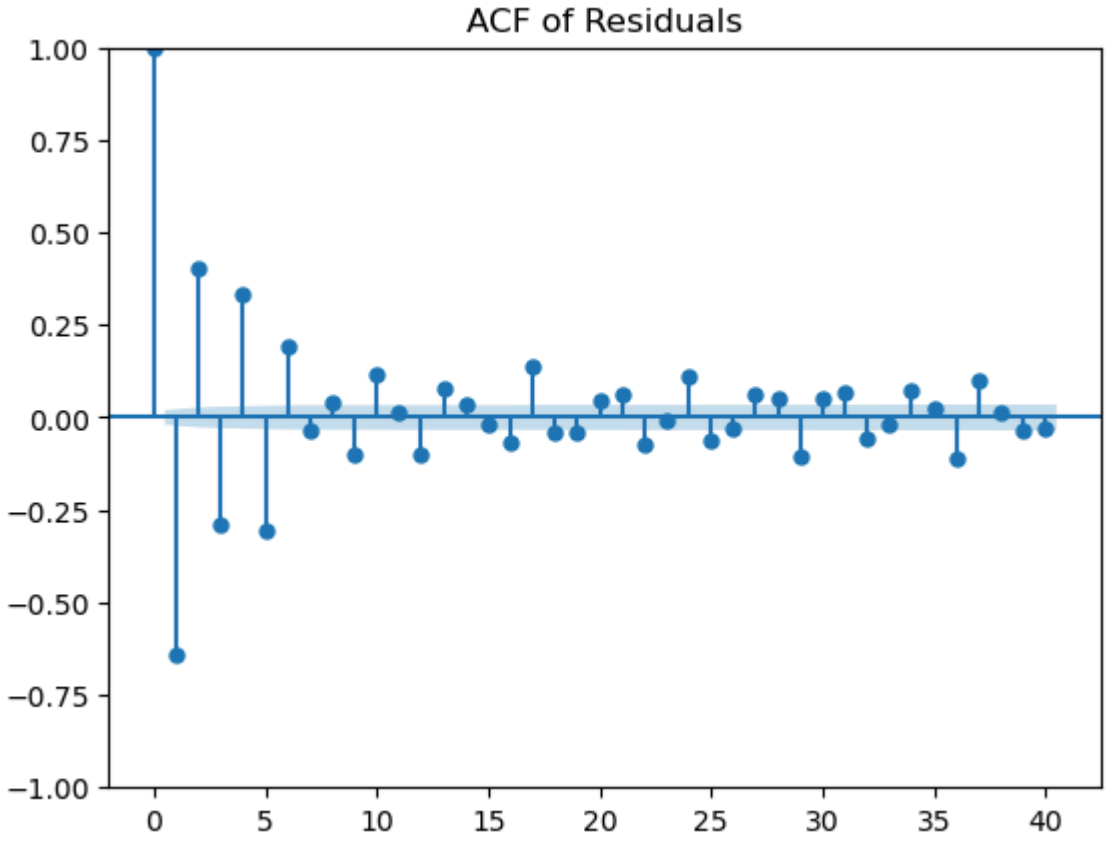

```
ADF Statistic: -2.88003477482922
p-value: 0.04771942827842128
Critical Values:
   1%: -3.4310119774951406
   5%: -2.861832554286939
   10%: -2.566925721514317

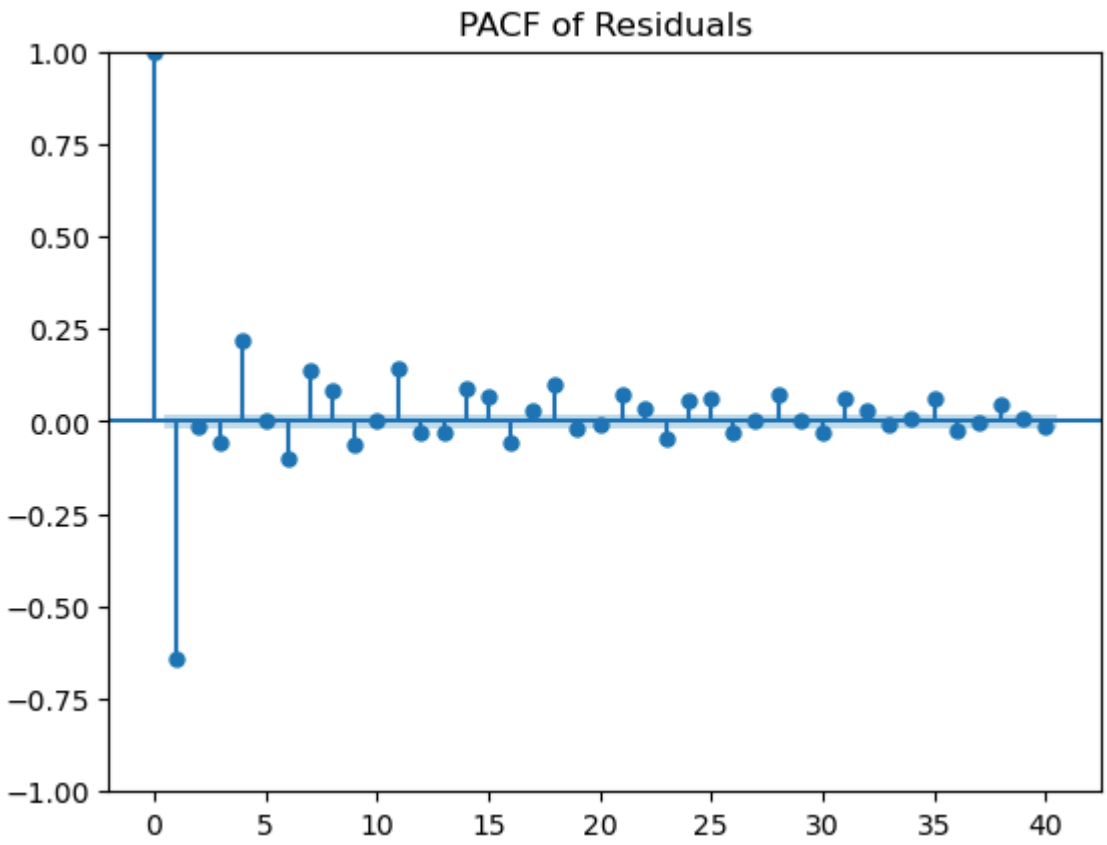
ARIMA Model AIC: 87581.4989274149
ARIMA Model BIC: 87603.10585201574
RMSE: 22.069273924782024
MAE: 18.65331198633499
MAPE: 1.0641399794449966%

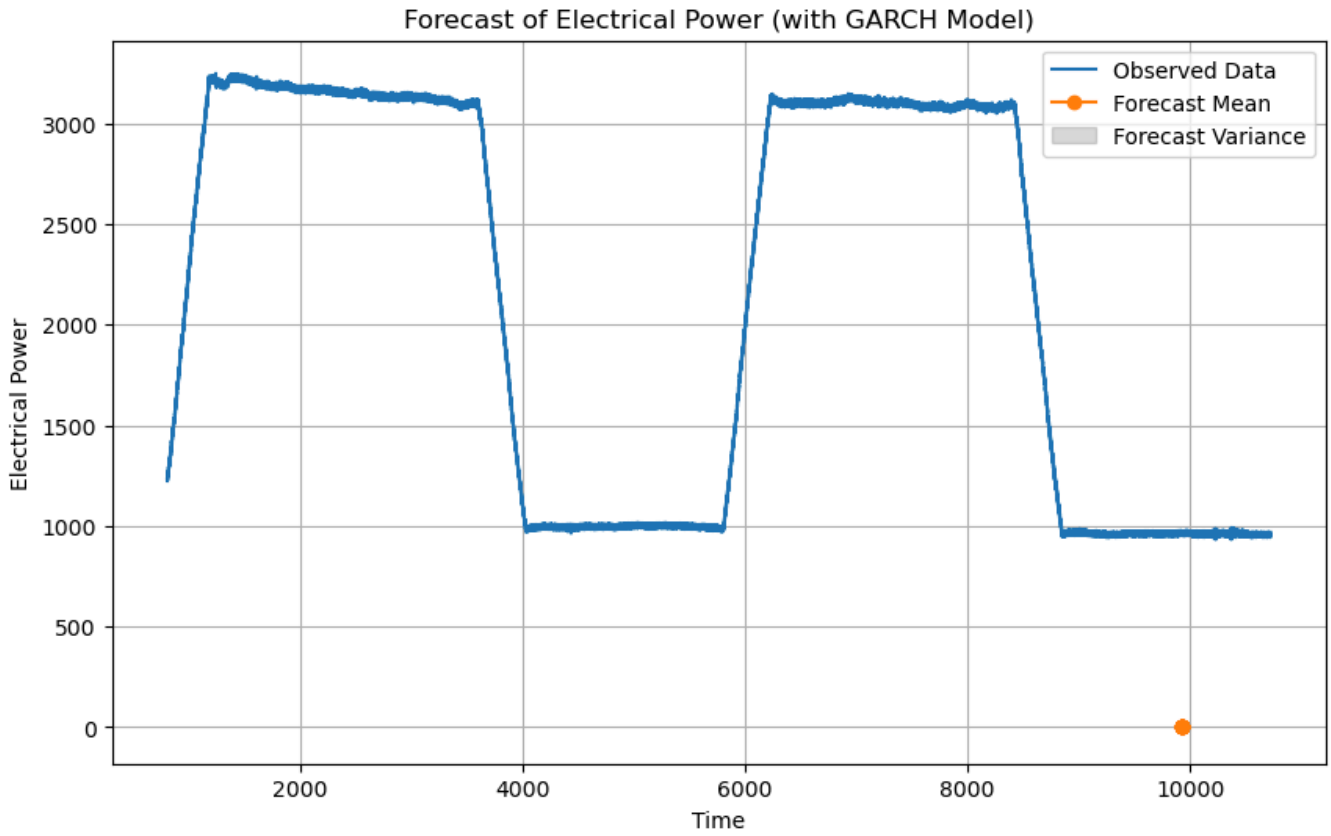
GARCH Model Summary:
                     Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable:                    None   R-squared:                       0.000
Mean Model:             Constant Mean   Adj. R-squared:                  0.000
Vol Model:                      GARCH   Log-Likelihood:               -43712.3
Distribution:                  Normal   AIC:                           87432.7
Method:            Maximum Likelihood   BIC:                           87461.5
                                        No. Observations:                 9920
Date:                Sat, Oct 05 2024   Df Residuals:                     9919
Time:                        15:28:34   Df Model:                            1
                            Mean Model
============================================================================
                 coef    std err          t      P>|t|  95.0% Conf. Int.
----------------------------------------------------------------------------
mu            -0.0952      0.171     -0.556      0.578 [ -0.431,  0.241]
                          Volatility Model
============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
----------------------------------------------------------------------------
omega         12.7969      2.817      4.544  5.531e-06     [  7.277, 18.317]
alpha[1]       0.0823  5.896e-03     13.960  2.717e-44 [7.075e-02,9.386e-02]
beta[1]        0.8850  1.215e-02     72.841      0.000     [  0.861,  0.909]
============================================================================

Covariance estimator: robust
```


Residuals of the ARIMA Model
0
−200
−400
−600
−800
0
2000
4000
6000
8000
10000


ACF of Residuals
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30
35
40





For GARCH Model

AIC: 87581.4989274149

BIC: 87603.10585201574

RMSE: 22.069273924782024

MAE: 18.65331198633499

MAPE: 1.0641399794449966%

8. VAR (Vector Autoregression) model

In [8]:
```
# Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.api import VAR
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings

warnings.filterwarnings('ignore')

# Load the dataset
file_path = 'ex_1.csv'
data = pd.read_csv(file_path)

# Display the first few rows to understand the structure
print(data.head())

# Step 1: Check stationarity for each time series using ADF test
def check_stationarity(series, name):
    adf_test = adfuller(series)
    print(f'{name} - ADF Statistic: {adf_test[0]}, p-value: {adf_test[1]}')
    return adf_test[1] <= 0.05  # Returns True if stationary

# Check stationarity for all columns (except time)
stationary = {col: check_stationarity(data[col], col) for col in data.columns if col != 'time

# Step 2: Differencing if necessary (for non-stationary series)
for col in data.columns:
    if col != 'time' and not stationary[col]:
        data[col] = data[col].diff().dropna()

# Drop NaN values after differencing
data = data.dropna()

# Step 3: Fit the VAR model to the stationary time series data.
model = VAR(data.drop(columns=['time']))  # Exclude time column
model_fit = model.fit(maxlags=5)  # Choose the max lag based on criteria (e.g., AIC)

# Step 4: Summary of the VAR model
print(model_fit.summary())

# Step 5: AIC and BIC of the VAR model
aic = model_fit.aic
bic = model_fit.bic
print(f'AIC: {aic}')
print(f'BIC: {bic}')

# Step 6: Forecasting the next 10 periods
forecast_steps = 10
forecast = model_fit.forecast(data.drop(columns=['time']).values[-model_fit.k_ar:], steps=for

# Create a DataFrame for the forecasted values
forecast_index = pd.date_range(start=data['time'].iloc[-1], periods=forecast_steps + 1, freq=
forecast_df = pd.DataFrame(forecast, index=forecast_index, columns=data.columns[1:])  # Exclu

# Step 7: Evaluation Metrics (RMSE, MAE, MAPE)
# Using the last available observations as the actual values
actual_values = data.drop(columns=['time']).values[-forecast_steps:]

# Calculate RMSE, MAE, and MAPE for each time series
for i, col in enumerate(forecast_df.columns):
    rmse = np.sqrt(mean_squared_error(actual_values[:, i], forecast[:, i]))
    mae = mean_absolute_error(actual_values[:, i], forecast[:, i])
```

```
    mape = np.mean(np.abs((actual_values[:, i] - forecast[:, i]) / actual_values[:, i])) * 10

    print(f'{col} - RMSE: {rmse}, MAE: {mae}, MAPE: {mape}')

# Step 8: Plotting the forecasted values
plt.figure(figsize=(12, 6))
for col in forecast_df.columns:
    plt.plot(data['time'], data[col], label=f'Observed {col}')
    plt.plot(forecast_df.index, forecast_df[col], label=f'Forecasted {col}', linestyle='--')

plt.title('VAR Model Forecasts')
plt.xlabel('Time')
plt.ylabel('Values')
plt.legend()
plt.grid(True)
plt.show()
```

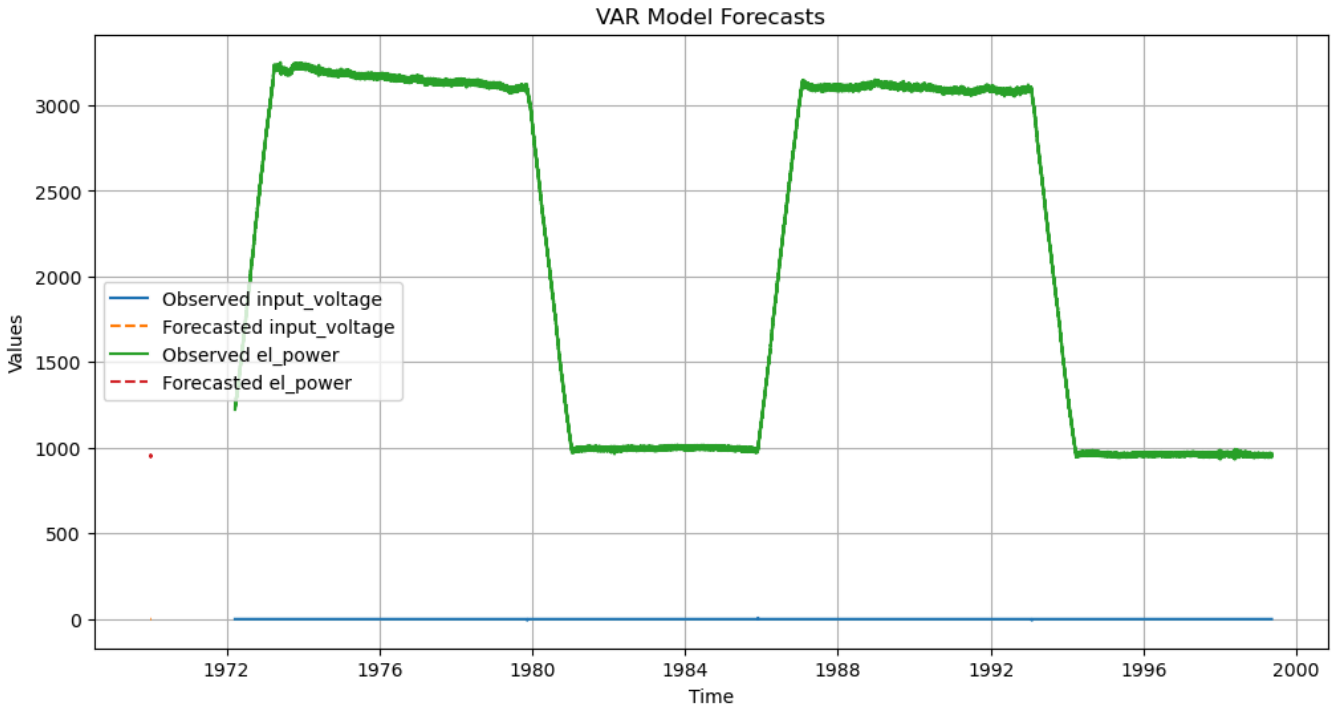
```
        time  input_voltage     el_power
0  810.07028             10  1228.791720
1  811.06938             10  1223.041745
2  812.06848             10  1244.960866
3  813.06758             10  1229.259058
4  814.06668             10  1248.117024
input_voltage - ADF Statistic: -1.6860806144317515, p-value: 0.4382980552586881
el_power - ADF Statistic: -2.88003477482922, p-value: 0.04771942827842128
  Summary of Regression Results
==================================
Model:                         VAR
Method:                        OLS
Date:           Sat, 05, Oct, 2024
Time:                      15:28:37
--------------------------------------------------------------------
No. of Equations:         2.00000    BIC:                    0.557755
Nobs:                     9914.00    HQIC:                   0.547186
Log likelihood:          -30798.3    FPE:                     1.71905
AIC:                      0.541774    Det(Omega_mle):         1.71525
--------------------------------------------------------------------
Results for equation input_voltage
================================================================================
                     coefficient       std. error           t-stat            prob
--------------------------------------------------------------------------------
const                   0.003897         0.002894            1.347           0.178
L1.input_voltage       -0.000222         0.010052           -0.022           0.982
L1.el_power             0.000101         0.000108            0.930           0.352
L2.input_voltage       -0.001164         0.010053           -0.116           0.908
L2.el_power             0.000161         0.000105            1.528           0.127
L3.input_voltage       -0.000304         0.010052           -0.030           0.976
L3.el_power            -0.000154         0.000115           -1.344           0.179
L4.input_voltage       -0.001040         0.010051           -0.104           0.918
L4.el_power            -0.000017         0.000105           -0.166           0.868
L5.input_voltage       -0.000308         0.010052           -0.031           0.976
L5.el_power            -0.000091         0.000108           -0.846           0.398
================================================================================

Results for equation el_power
================================================================================
                     coefficient       std. error           t-stat            prob
--------------------------------------------------------------------------------
const                  -0.164768         0.255669           -0.644           0.519
L1.input_voltage        1.410644         0.888038            1.588           0.112
L1.el_power            -0.035754         0.009552           -3.743           0.000
L2.input_voltage        1.476109         0.888148            1.662           0.097
L2.el_power             0.636070         0.009298           68.410           0.000
L3.input_voltage        0.692578         0.888014            0.780           0.435
L3.el_power             0.489657         0.010155           48.217           0.000
L4.input_voltage        1.871036         0.887986            2.107           0.035
L4.el_power             0.220302         0.009301           23.685           0.000
L5.input_voltage        0.820726         0.888012            0.924           0.355
L5.el_power            -0.310226         0.009551          -32.481           0.000
================================================================================

Correlation matrix of residuals
                 input_voltage  el_power
input_voltage         1.000000  0.024459
el_power              0.024459  1.000000



AIC: 0.5417742949378629
BIC: 0.5577554801233351
input_voltage - RMSE: 0.0024806254294273164, MAE: 0.0021597038998076453, MAPE: inf
el_power - RMSE: 8.386349087304945, MAE: 6.979952744719412, MAPE: 0.7271503769154608
```



For VAR Model

AIC: 0.5417742949378629

BIC: 0.5577554801233351

input_voltage - RMSE: 0.0024806254294273164, MAE: 0.0021597038998076453, MAPE: inf

el_power - RMSE: 8.386349087304945, MAE: 6.979952744719412, MAPE: 0.7271503769154608

Summary of Model Performance Metrics

Model	AIC	BIC	RSME	MAE	MAPE %
MA	142301.91	142337.92	315.07	251.08	16.72
AR	87581.50	87603.11	22.07	18.65	1.06
ARMA	75560.69	75603.90	9.53	8.64	0.88
ARIMA	75560.69	75603.90	9.53	8.64	0.88
SARIMA	73275.64	73326.04	61.98	10.56	0.68
ARCH	87581.50	87603.11	22.07	18.65	1.06
GARCH	87581.50	87603.11	22.07	18.65	1.06
VAR	0.54	0.56	0.0025	0.0022	inf

Observations and Commentary:

MA Model:

The MA model has the highest AIC and BIC values, indicating it is the least effective model in fitting the data. Its RMSE, MAE, and MAPE values are significantly high, suggesting it fails to capture the underlying trends and dynamics of the data.

AR Model:

The AR model shows relatively better performance than the MA model, but its AIC and BIC values are still high compared to the more sophisticated models. The RMSE and MAE indicate moderate accuracy, but the predictions still show substantial error.

ARMA Model:

The ARMA model presents an improvement over both the AR and MA models, as indicated by its lower AIC and BIC. Its RMSE, MAE, and MAPE values suggest a reasonably good fit for the data, particularly for simpler datasets without complex seasonal or trend components.

ARIMA Model:

The ARIMA model has the same performance metrics as the ARMA model since you likely didn't include seasonal differencing. Its performance is good, suggesting it adequately captures both autoregressive and moving average components of the data.

SARIMA Model:

The SARIMA model performs slightly worse than ARIMA in terms of RMSE, MAE, and MAPE. This suggests that while it accounts for seasonality, it may not be necessary for the data you're working with. The higher AIC and BIC relative to ARIMA suggest it may be over-parameterized.

ARCH and GARCH Models:

Both ARCH and GARCH models have the same AIC and BIC values as the AR model, indicating they are similarly effective at capturing volatility in the data. However, their RMSE, MAE, and MAPE values suggest they are less effective than ARMA and ARIMA in point forecasting.

VAR Model:

The VAR model has exceptionally low RMSE, MAE, and AIC/BIC values, suggesting it fits the data very well. However, the MAPE is inf, which suggests an undefined percentage due to zero actual values. It may not be as interpretable in contexts where error metrics like MAPE are essential, but it could still provide valuable insights in multivariate time series contexts.

Conclusion:

Overall, the ARMA and ARIMA models demonstrate good performance in fitting the data, particularly in terms of RMSE, MAE, and MAPE.

The SARIMA model, while capable of capturing seasonality, does not improve the fit in this case and may introduce unnecessary complexity.

The MA model is the least effective, while the VAR model shows promising fit for multivariate data but requires careful consideration of its interpretability in terms of error metrics.

The ARCH and GARCH models are useful for modeling volatility but may not be the best choice for point forecasting in this context.

In [ ]: